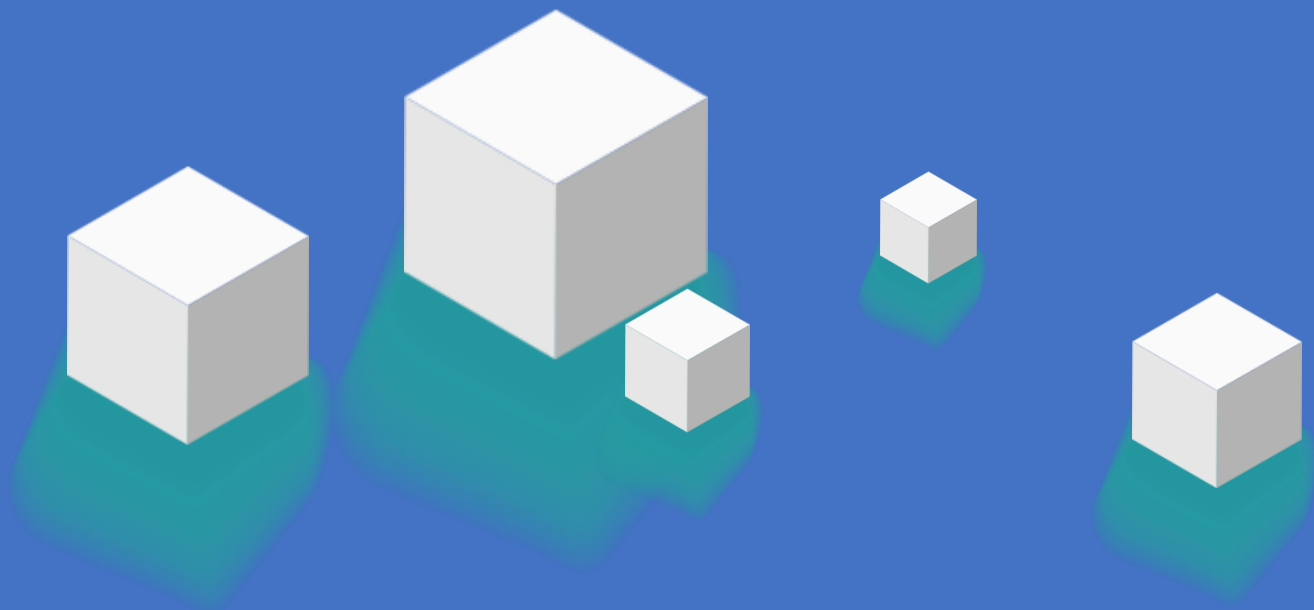


SGLang 深度优化:

Radix 缓存与复杂任务的极致吞吐



>> 今天的学习目标

SGLang 深度优化

- SGLang vs vLLM
- PD 分离架构 -- Prefill/Decode 解耦
- SGLang语言
- SGLang编程示例
- SGLang系统中的RadixAttention技术
- 缓存感知调度
- 约束解码的需求
- API推测执行
- SGLang使用
- 常见启动参数
- 常见问题排查
- 场景1：基础对话
- 场景2：流式输出
- 场景3：多轮对话
- 场景4：批量任务处理
- 场景5：JSON 约束输出
- 性能验证与监控
- vLLM vs SGLang 性能对比实战

SGLang vs vLLM

vLLM 的高并发原理:

PagedAttention (解决 KV Cache 碎片)

Continuous Batching (解决 GPU 空闲)

推测解码 (加速 Decode 阶段)

=> vLLM 解决的是 如何让模型跑得快 的问题。

Thinking: SGLang的作用是什么?

解决的是 如何让业务逻辑跑得快

SGLang vs vLLM

维度	vLLM	SGLang	选型建议
定位	通用推理引擎（像 MySQL）	复杂 LLM 程序运行时（像 Redis+Lua）	简单对话用 vLLM；Agent/多轮/结构化输出用 SGLang
缓存策略	PagedAttention（块级复用）	RadixAttention（前缀树复用）	多轮对话场景 SGLang 缓存命中率更高
编程方式	OpenAI API 调用	Python 原生嵌入（@function）	需要复杂控制流（分支、循环、并行）选 SGLang
JSON 生成	靠 Prompt + 后校验	原生约束解码（正则强制）	必须输出标准 JSON 的接口用 SGLang 更稳
兼容性	OpenAI API 兼容	OpenAI API 兼容 + SGLang 原生语法	SGLang 也支持 OpenAI API，迁移成本低

vLLM 是跑模型的，SGLang 是跑业务逻辑的。

简单问答 => 用 vLLM 就够了； 多轮对话、并行任务、严格 JSON 输出 => 用 SGLang 省钱又稳定。

PD 分离架构 -- Prefill/Decode 解耦

PD 分离架构

PD分离（Prefill-Decode Decoupling）是 LLM 推理优化中的一种架构设计。

1. Prefill（预填充）是什么？

Prefill 是模型处理用户输入（Prompt）的阶段。

- 做什么：一次性读入用户的整个问题/提示，计算所有 token 的 KV Cache（键值缓存）
- 计算特点：计算密集型（Compute-Bound）——大量矩阵乘法，GPU 算力是瓶颈
- 关键指标：TTFT（Time To First Token，首 token 延迟）
- 资源需求：需要高算力、高并行度

=> 像短跑运动员，爆发力强，瞬间完成大量计算

PD 分离架构

2. Decode（解码）是什么？

Decode 是模型生成回答（逐个吐字）的阶段。

- 做什么：基于已生成的内容，逐个预测下一个 token（自回归生成）
- 计算特点：内存带宽密集型（Memory-Bound）——每步只算 1 个 token，瓶颈在读取 KV Cache，GPU 计算单元大量空闲
- 关键指标：TPOT（Time Per Output Token，token 间延迟）
- 资源需求：需要大显存、高内存带宽

=> 像马拉松运动员，持久耐力，一步步往前跑

为什么需要 PD 分离?

Continuous Batching 将 Prefill（预填充）和 Decode（解码）混合在同一个 GPU 上执行。

虽然提升了吞吐，但两个阶段的计算特性完全不同，混在一起会产生严重的资源争抢。

对比维度	Prefill（预填充）	Decode（解码）
做什么	一次性处理整个 Prompt，计算所有 token 的 KV Cache	逐个生成新 token（自回归）
计算特性	计算密集型 (Compute-Bound) 大量矩阵乘法，GPU 算力是瓶颈	内存带宽密集型 (Memory-Bound) 每步只算 1 个 token，瓶颈在读取 KV Cache
资源需求	高算力、高并行度	大显存、高带宽
关键指标	TTFT（首 Token 延迟）	TPOT（Token 间延迟）

为什么需要 PD 分离?

Thinking: 混合执行的问题是什么?

当 Prefill 和 Decode 在同一个 GPU 上交替执行时:

- 执行计算密集的 Prefill 时, 会阻塞延迟敏感的 Decode → TPOT 飙升 (用户感觉卡顿)
- 执行访存密集的 Decode 时, GPU 计算单元大量空闲 → 浪费了 Prefill 需要的算力

=> 让短跑运动员和马拉松运动员在同一条跑道上交替训练, 谁都练不好。

PD 分离的原理

PD 分离架构工作流程:

用户请求 (Prompt)

Router (路由层)

将新请求分发到 Prefill 节点

1 Prefill 集群 (计算密集型 GPU)

专门处理 Prompt 的全序列注意力计算
生成完整的 KV Cache

适合高算力 GPU (如 H100)

小 Batch

2 KV Cache 传输

通过高速网络 (NVLink / RDMA / InfiniBand)
将 KV Cache 从 Prefill 节点迁移到 Decode 节点

3 Decode 集群 (带宽密集型 GPU)

专门执行自回归生成, 利用高带宽快速读取 KV Cache
可进一步叠加推测解码 (EAGLE/MEDUSA)

适合大显存 GPU (如 A100 80G)

大 Batch

PD 分离的核心优势

- 消除干扰

两阶段物理隔离，Prefill 不再阻塞 Decode，TPOT（首Token延迟）稳定

- 资源特化

Prefill 用高算力 GPU，Decode 用大显存 GPU，各取所需

- 独立扩展

根据负载动态调整 P/D 实例比例，灵活应对流量变化

- Batching 差异化

Prefill 用小 Batch（已是 Compute-Bound），Decode 用大 Batch（缓解 Memory-Bound）

PD 分离的核心优势

Thinking: 为什么 Prefill 适合小 Batch, Decode 适合大 Batch?

- Prefill 本身就是计算密集型（处理整个 Prompt 的矩阵乘法），GPU 算力已经打满了。

增大 Batch 只会增加排队延迟，不提升吞吐。

- Decode 本身是内存带宽密集型（每步只算 1 个 token），GPU 算力大量空闲。

增大 Batch 让 GPU 同时处理更多请求，空闲的算力被利用起来，从 Memory-Bound 转向 Compute-Bound，吞吐大幅提升。

vLLM 中的 PD 分离实现

vLLM 通过 KV Transfer Connector 机制实现 PD 分离：

```
# Prefill 节点配置 (生产者 -- 负责计算 KV Cache)
kv_config = KVTransferConfig.from_cli(
    '{"kv_connector": "PyNcclConnector", "kv_role": "kv_producer", "kv_rank": 0}'
)

# Decode 节点配置 (消费者 -- 接收 KV Cache 并生成)
kv_config = KVTransferConfig.from_cli(
    '{"kv_connector": "PyNcclConnector", "kv_role": "kv_consumer", "kv_rank": 1}'
)
```

- 异步传输：

Prefill 计算完成后通过 Connector 将 KV Cache 推送到 Decode 节点

- 零拷贝传输：

支持通过 NVLink、InfiniBand 等高速互联，减少传输开销

- 调度协同：

Router 决定请求在本地 Prefill 还是远程执行

- 兼容性：

PD 分离架构下仍然可以使用 Continuous Batching 分别处理各自的请求队列

vLLM 中的 PD 分离实现

Thinking: 怎么理解 vLLM 通过 KV Transfer Connector 机制实现 PD 分离 的场景?

节点	角色	职责
Prefill 节点	Producer (生产者)	中央厨房负责切菜、备料 (计算 KV Cache)
Decode 节点	Consumer (消费者)	门店服务员负责上菜、服务顾客 (生成 token)

顾客点了宫保鸡丁 (Prompt)

厨房 (Prefill)：切鸡肉、剥花生、调酱汁 → 准备好所有食材 (KV Cache)

服务员 (Decode)：拿着备好的料，一勺一勺盛给客人 (逐个生成 token)

PyNcclConnector 相当于专用传菜电梯

=> 厨房一口气备好 10 份菜品 (KV Cache)，通过 Connector 推送给 Decode

Decode 一边接收，一边开始给客人上菜

Summary

除了 vLLM，业界还有其他 PD 分离实现：

- Mooncake（Kimi 的推理架构）：以 KV Cache 为中心的分离式架构
- NVIDIA Dynamo：提供声明式的 PD 分离部署（VllmPrefillWorker + VllmDecodeWorker）
- SGLang、llm-d（Kubernetes 原生推理栈）

Summary

Thinking: PagedAttention, Continuous Batching, 推测解码, PD分离 分别解决什么问题?

- PagedAttention 解决显存浪费
- Continuous Batching 解决 GPU 空闲
- 推测解码 解决生成太慢
- PD 分离 解决资源争抢。

Summary

- PagedAttention

优化层面：内存层

解决的问题：KV Cache 碎片和浪费

=> 把 KV Cache 切成小块，按需分配，像虚拟内存一样管理

- Continuous Batching

优化层面：调度层

解决的问题：GPU 空闲等待（短请求完成后干等长请求）

=> 每生成一个 token 就调度一次，完成即腾位，新请求立即填入

Summary

- 推测解码 (EAGLE/MEDUSA)

优化层面：算法层

解决的问题：Decode 阶段 GPU 算力空闲(Memory-Bound)

=> 小模型猜多个 token，大模型一次并行验证，减少生成步数

- PD 分离

优化层面：架构层

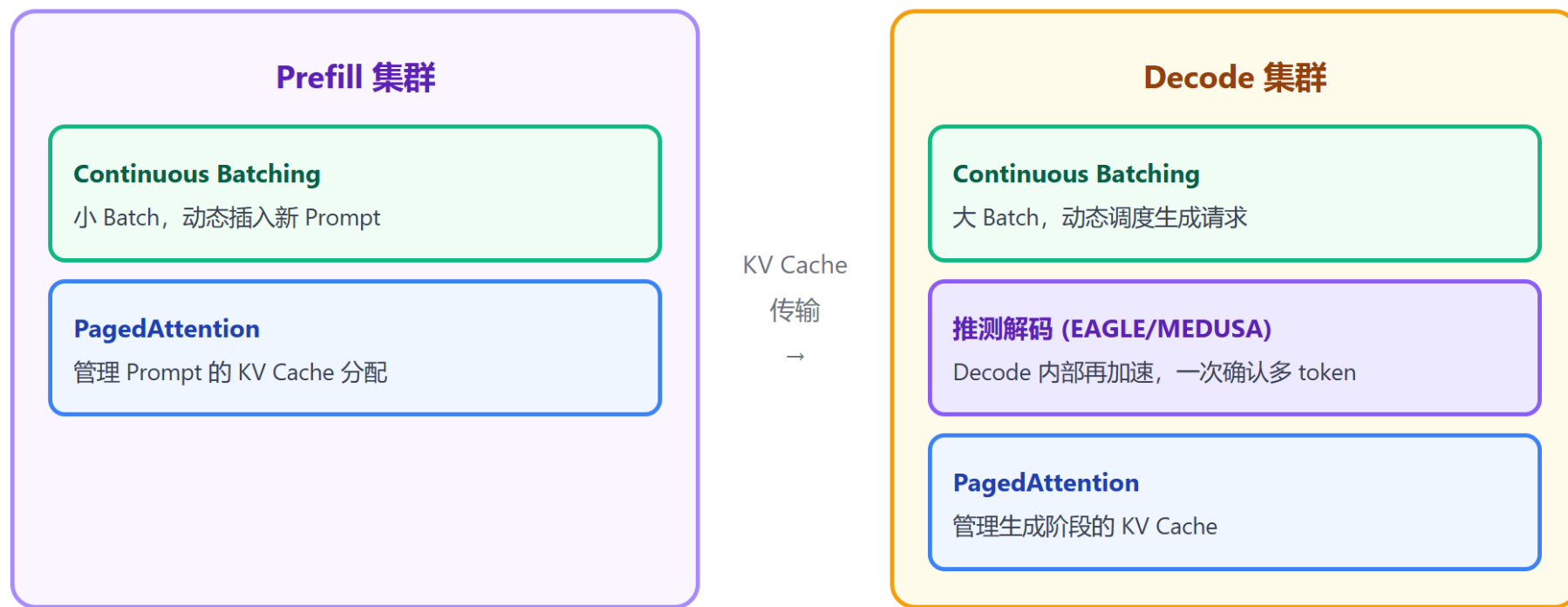
解决的问题：Prefill 和 Decode 互相干扰

=> 两阶段物理隔离到不同 GPU，各自用最适合的硬件和 Batch 策略

Summary

Thinking: PagedAttention, Continuous Batching, 推测解码, PD分离 如何叠加使用?

PD 分离 (架构层) -- 最外层的部署策略



Summary

使用逻辑：

- PagedAttention 是基础设施，贯穿 Prefill 和 Decode 两端，所有场景都需要
- Continuous Batching 在 Prefill 和 Decode 集群内部各自运行，提升各自的 GPU 利用率
- 推测解码 只在 Decode 集群中使用（Decode 阶段是 Memory-Bound，最需要加速）
- PD 分离 是最外层的部署策略，决定了上面三者在哪里运行

Summary

技术	vLLM 支持状态	启用方式
PagedAttention	默认启用	vLLM 核心机制，无需额外配置
Continuous Batching	默认启用	Iteration-level 调度自动运行
EAGLE 推测解码	已官方集成	--speculative-model <eagle_model>
MEDUSA 推测解码	社区支持	--speculative-model <medusa_model>
PD 分离	已支持	KV Transfer Connector 配置 P/D 角色

在vLLM中的应用

SGLang深度优化

从LLM Inference到LLM Programs

LLM互动的方式：

- 从简单的聊天 => 更复杂的程序化使用LLMs
- LM程序：使用程序来调度和控制LLMs的生成过程，将这些程序称为“语言模型程序”

LM程序通常有两个共同属性：

1) LM程序通常包含多个LLM调用，穿插控制流。

=> 完成复杂任务和提高整体质量所必需的。

2) LM程序接收结构化输入并产生结构化输出。

=> 为了使LM程序能够组合，并集成到现有的软件系统中。

现在的 LLM 应用已经从聊天进化到LLM Programs

vLLM 解决的是模型跑得快，但 SGLang 解决的是业务逻辑跑得快。

SGLang语言

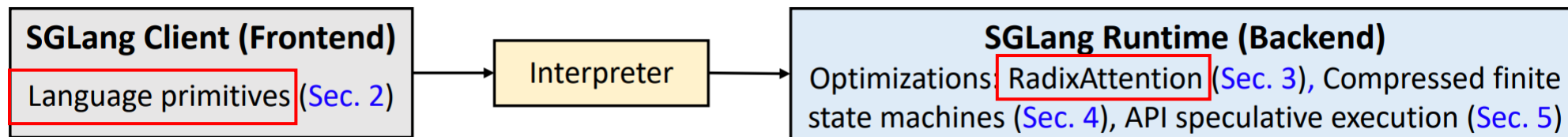
SGLang:

一种为LLMs设计的结构化生成语言，用于简化和加速复杂语言模型程序的编程和执行。

SGLang由前端语言和后端运行时两部分组成，分别负责简化编程和加速执行。

实验表明，SGLang在各种大型语言和多模态模型上的任务中，实现了高达6.4倍的吞吐量提升。

<https://github.com/sgl-project/sglang>



SGLang架构：解释器通过优化的运行时，执行语言原语

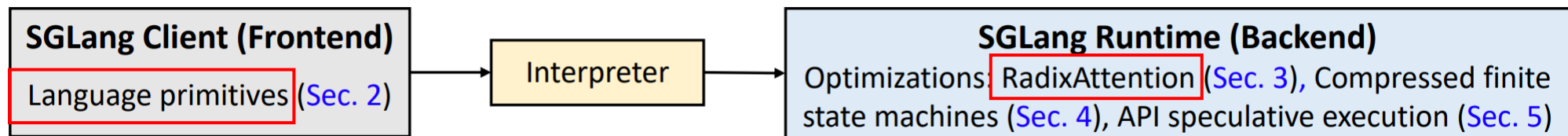
SGLang语言

SGLang的编程简化:

- SGLang作为嵌入在Python中的领域特定语言，提供了易于使用的原语，如生成控制（`gen`、`select`）和并行控制（`fork`、`join`）。
- 它与Python兼容，开发复杂的提示 workflow 更加容易。

SGLang的运行时优化技术:

- **RadixAttention**: 允许在多个生成调用之间自动重用KV缓存，从而减少不必要的计算和内存浪费。
- **压缩有限状态机**: 通过构建压缩的有限状态机来实现更快的约束解码，允许一次性解码多个标记。
- **API推测执行**: 针对仅通过API访问的模型，SGLang通过推测执行来优化多调用程序，减少API调用次数和成本。



系统架构:解释器通过优化的运行时执行语言原语

SGLang编程示例

SGLang是一种嵌入在Python中的领域特定语言，开发者可以使用Python来编写。

多维论文评分程序：评估关于图像的文章。
在SGLang中使用了 分支-解决-合并的技术，
将一个大问题分解成几个小问题，分别解决，
然后再合并起来。

函数参数：multi_dimensional_judge函数接受三个参数：

s: 管理提示状态

path: 图像文件路径

essay: 文章文本

```
dimensions = ["Clarity", "Originality", "Evidence"]
```

```
@function
```

```
def multi_dimensional_judge(s, path, essay):
```

```
    s += system("Evaluate an essay about an image.")
```

```
    s += user(image(path) + "Essay:" + essay)
```

```
    s += assistant("Sure!")
```

```
    # Return directly if it is not related
```

```
    s += user("Is the essay related to the image?")
```

```
    s += assistant(select("related", choices=["yes", "no"]))
```

```
    if s["related"] == "no": return
```

```
    # Judge multiple dimensions in parallel
```

```
    forks = s.fork(len(dimensions))
```

```
    for f, dim in zip(forks, dimensions):
```

```
        f += user("Evaluate based on the following dimension:" +
```

```
                  dim + ". End your judgment with the word 'END'")
```

```
        f += assistant("Judgment:" + gen("judgment", stop="END"))
```

```
    # Merge the judgments
```

```
    judgment = "\n".join(f["judgment"] for f in forks)
```

```
    # Generate a summary and a grade. Return in the JSON format.
```

```
    s += user("Provide the judgment, summary, and a letter grade")
```

```
    s += assistant(judgment + "In summary," + gen("summary", stop=".")
```

```
                  + "The grade of it is" + gen("grade"))
```

```
    schema = r'{"summary": "[\w\d\s]+\.", "grade": "[ABCD][+-]?"}'
```

```
    s += user("Return in the JSON format.")
```

```
    s += assistant(gen("output", regex=schema))
```

```
state = multi_dimensional_judge.run(...)
```

```
print(state["output"])
```

Handle chat template
and multi-modal inputs

Select an option with
the highest probability

Fetch result; Use Python
control flow

Runtime optimization:
KV Cache Reuse (Sec. 3)

Multiple generation
calls run in parallel

Fetch generation results

Runtime optimization: API
speculative execution (Sec. 5)

Runtime optimization: fast
constrained decoding (Sec. 4)

Run an SGLang program

SGLang 中多维度论文评判的实现使用了分支-求解-合并的提示技术
SGLang 提供的原语以红色标出。

SGLang编程示例

语言原语： SGLang提供了一组原语，用于控制语言模型的提示状态

Python兼容性： 这些原语可以与Python语法和库一起使用

function： 定义一个函数。

system： 表示系统级别的操作。

user： 代表用户的输入或操作。

image： 处理图像输入的原语。

assistant： 代表助手的回应或输出。

select： 从一组选项中选择最高概率的选项。

fork： 创建并行处理的分支。

gen： 生成文本或内容的操作。

run： 执行或运行一段代码。

```
dimensions = ["Clarity", "Originality", "Evidence"]
@function
def multi_dimensional_judge(s, path, essay):
    s += system("Evaluate an essay about an image.")
    s += user(image(path) + "Essay:" + essay)
    s += assistant("Sure!")
    # Return directly if it is not related
    s += user("Is the essay related to the image?")
    s += assistant(select("related", choices=["yes", "no"]))
    if s["related"] == "no": return
    # Judge multiple dimensions in parallel
    forks = s.fork(len(dimensions))
    for f, dim in zip(forks, dimensions):
        f += user("Evaluate based on the following dimension:" +
            dim + ". End your judgment with the word 'END'")
        f += assistant("Judgment:" + gen("judgment", stop="END"))
    # Merge the judgments
    judgment = "\n".join(f["judgment"] for f in forks)
    # Generate a summary and a grade. Return in the JSON format.
    s += user("Provide the judgment, summary, and a letter grade")
    s += assistant(judgment + "In summary," + gen("summary", stop=".")
        + "The grade of it is" + gen("grade"))
    schema = r'{"summary": "[\w\d\s]+\.", "grade": "[ABCD][+-]?\"}'
    s += user("Return in the JSON format.")
    s += assistant(gen("output", regex=schema))
state = multi_dimensional_judge.run(...)
print(state["output"])
```

Handle chat template and multi-modal inputs

Select an option with the highest probability

Fetch result; Use Python control flow

Runtime optimization: KV Cache Reuse (Sec. 3)

Multiple generation calls run in parallel

Fetch generation results

Runtime optimization: API speculative execution (Sec. 5)

Runtime optimization: fast constrained decoding (Sec. 4)

Run an SGLang program

SGLang 中多维度论文评判的实现使用了分支-求解-合并的提示技术
SGLang 提供的原语以红色标出。

SGLang编程示例

`dimensions = ["Clarity", "Originality", "Evidence"]`: 包含了将要评估文章三个维度: 清晰度、原创性和证据支持。

`@function def multi_dimensional_judge(s, path, essay):`: 定义了一个 SGLang 函数 `multi_dimensional_judge`, 它接受三个参数: `s` (状态), `path` (图片路径), 和 `essay` (文章)。

3-5. 函数开始时, 向状态 `s` 中添加系统指令和用户请求, 以及助手 (`assistant`) 的响应。

6-11. 检查文章是否与图片相关。如果不相关, 则函数返回。

12-16. 通过 `fork` 函数创建多个并行流, 每个流评估文章的不同维度。合并各个维度的评估结果。

18-22. 生成总结和评分, 并要求以 JSON 格式返回结果。

定义了正则表达式 `schema`, 用于确保输出符合特定的 JSON 格式。

24-25. 最后, 函数通过生成一个符合 JSON 格式的输出来结束。

调用 `multi_dimensional_judge.run(...)` 执行这个函数

打印出函数返回的 JSON 格式结果。

这个函数的关键在于并行处理多个评价维度, 然后合并结果, 并以 JSON 格式返回。SGLang 提供了一套原语 (如 `gen`, `select`, `fork`, `join` 等), 使得这种类型的生成任务更加高效。

```
dimensions = ["Clarity", "Originality", "Evidence"]

@function
def multi_dimensional_judge(s, path, essay):
    s += system("Evaluate an essay about an image.")
    s += user(image(path) + "Essay:" + essay)
    s += assistant("Sure!")

    # Return directly if it is not related
    s += user("Is the essay related to the image?")
    s += assistant(select("related", choices=["yes", "no"]))
    if s["related"] == "no": return

    # Judge multiple dimensions in parallel
    forks = s.fork(len(dimensions))
    for f, dim in zip(forks, dimensions):
        f += user("Evaluate based on the following dimension:" +
            dim + ". End your judgment with the word 'END'")
        f += assistant("Judgment:" + gen("judgment", stop="END"))

    # Merge the judgments
    judgment = "\n".join(f["judgment"] for f in forks)

    # Generate a summary and a grade. Return in the JSON format.
    s += user("Provide the judgment, summary, and a letter grade")
    s += assistant(judgment + "In summary," + gen("summary", stop=".")
        + "The grade of it is" + gen("grade"))

    schema = r'\{"summary": "[\w\d\s]+\.", "grade": "[ABCD][+-]?"\}'
    s += user("Return in the JSON format.")
    s += assistant(gen("output", regex=schema))

state = multi_dimensional_judge.run(...)
print(state["output"])
```

Handle chat template and multi-modal inputs

Select an option with the highest probability

Fetch result; Use Python control flow

Runtime optimization: KV Cache Reuse (Sec. 3)

Multiple generation calls run in parallel

Fetch generation results

Runtime optimization: API speculative execution (Sec. 5)

Runtime optimization: fast constrained decoding (Sec. 4)

Run an SGLang program

SGLang 中多维度论文评判的实现使用了分支-求解-合并的提示技术
SGLang 提供的原语以红色标出。

SGLang编程特点

- Python 原生兼容性

可以用熟悉的 `if/for/return` 写逻辑，不需要学习新的模板语法。比如示例中用 `if s["related"] == "no": return` 就能根据模型判断结果走不同分支。

- 隐式并行优化

`fork` 那段，表面上写了 `for` 循环，但运行时 SGLang 会让这 3 个维度的评判并行生成（通过 RadixAttention 复用共同的上下文）=> 这是手写 Prompt 很难做到的优化。

- 运行时自动加速

文档中右侧黄色箭头标注的优化点：

- 1) KV Cache 复用：多轮对话自动缓存前缀
- 2) 约束解码：`regex=schema` 确保 100% 合法 JSON，比 Prompt 工程靠谱
- 3) API 推测执行：即使是 OpenAI API 也能减少调用次数

SGLang编程

执行模式：

- 解释器模式

SGLang程序可以通过解释器执行，其中提示被视为一个异步流。允许程序在不等待生成完成的情况下继续运行。

- 编译器模式

SGLang程序也可以编译成计算图并由图执行器执行，这为更多的编译优化提供了可能性。

SGLang通过提供一套丰富的编程原语和执行模式，以及运行时优化技术，来简化和加速LLM程序的开发和执行。

- SGLang被归类为低级系统，与LMQL和Guidance类似。它允许直接操作提示和原语，而不改变提示本身。
- SGLang专注于运行时效率，并配有自己的运行时（SGLang Runtime, SRT）。

运行时优化

- **KV缓存重用**：通过在多个LLM调用之间重用KV缓存，减少不必要的计算和内存使用。
- **快速约束解码**：通过构建压缩的有限状态机，加快结构化输出（如JSON）的解码速度。
- **API推测执行**：针对API模型，如OpenAI的GPT-4，SGLang使用推测执行来优化多调用程序。

SGLang系统中的RadixAttention技术

RadixAttention技术:

一种用于高效重用LLM推理过程中的KVCache的方法

Thinking: LLM自回归解码中, KVCache重用的挑战

在LLM的自回归解码中, 每一步的输出都依赖于前一步的输出, 导致模型需要进行多次序列运行, 每次运行都涉及将完整的模型参数从高带宽内存(HBM)传输到加速器的缓存中 => 这种序列计算方式限制了模型的推理速度。

使用RadixAttention优化KVCache的重用:

- **基数树 (Radix Tree):** RadixAttention使用基数树来管理token序列及其对应的KV缓存张量的映射。基数树是一种高效的数据结构, 它允许快速的前缀搜索、重用、插入和逐出操作。
- **LRU逐出策略:** 系统实现了LRU (最近最少使用) 逐出策略, 优先逐出最久未使用的缓存条目。让常用的数据保持在缓存中。
- **缓存感知调度:** RadixAttention还包括一个缓存感知的调度策略, 它通过优化任务的执行顺序来提高缓存命中率, 从而减少不必要的计算和内存使用。

SGLang系统中的RadixAttention技术

Thinking: 怎么理解RadixAttention 是什么？ 解决什么问题？

问题：大模型生成文本时，每一步都要重复计算之前已经算过的内容（KV Cache），浪费算力和时间

解决方案：用基数树(Radix Tree) 管理token序列，自动重用之前计算好的缓存

配套机制：LRU（最近最少使用）淘汰策略 + 缓存感知调度

想象你是一个话剧团的道具管理员：

传统方式（无RadixAttention）：每次新场景上台，你都要重新从仓库拿全部道具，哪怕很多道具上一场刚用过

RadixAttention方式：

- 你有一个智能道具架（基数树），按照场景前缀分类
- 第一场戏是"皇宫-早上-宴会"，你拿出龙椅、酒杯、桌布
- 第二场戏是"皇宫-早上-议政"，"皇宫-早上"这部分道具已经在台上了，你只需要加一张书桌
- 如果道具架满了，你会把最久没使用的道具（LRU）收回仓库，但保留可能共用的基础布景

SGLang系统中的RadixAttention技术

基数树 = 智能分类货架，能快速找到"皇宫-早上"这个共同前缀

LRU淘汰 = 优先收走冷门道具，保留常用布景

缓存感知调度 = 导演合理安排上场顺序，让相似场景的戏连着拍，减少换道具时间

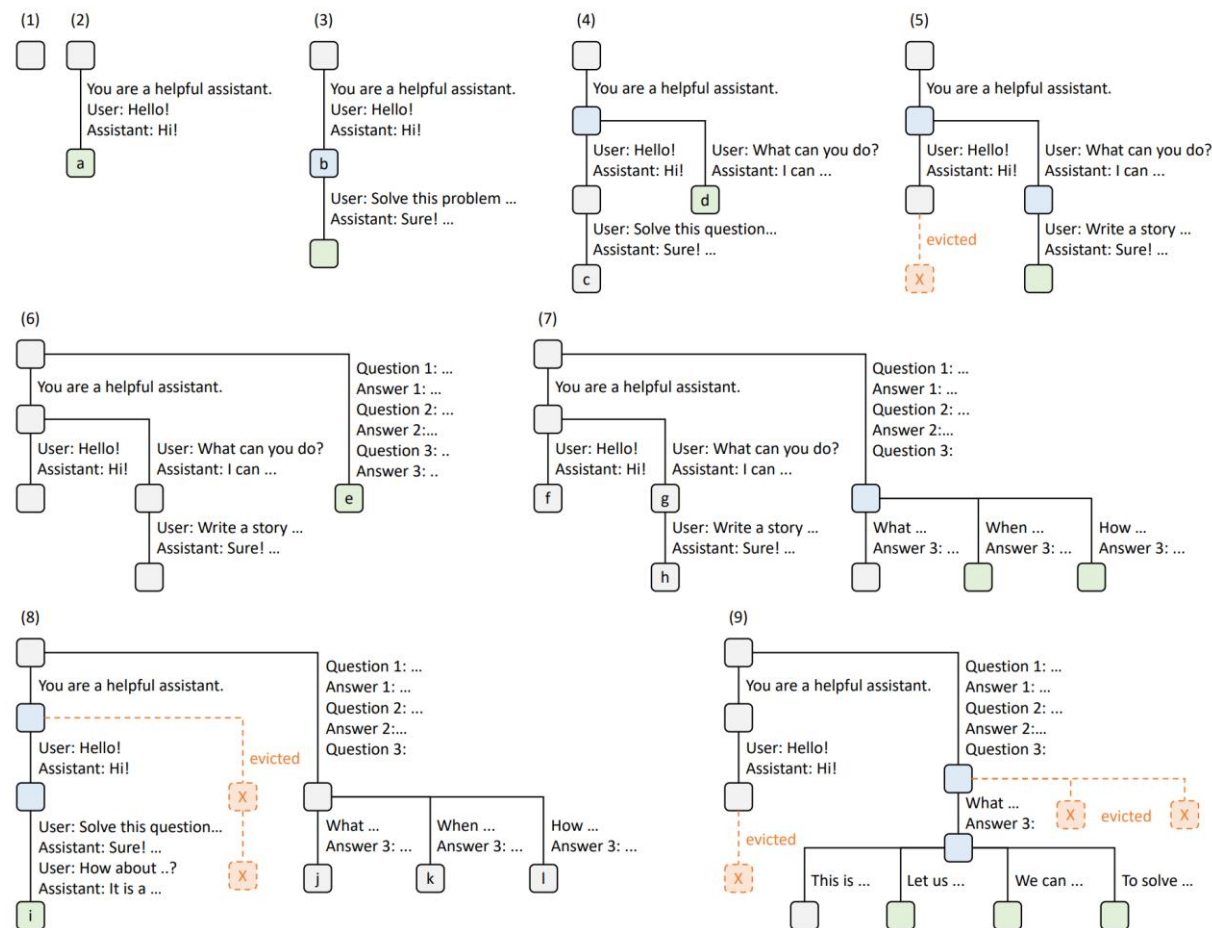
SGLang系统中的RadixAttention技术

RadixAttention:

基数树是一种数据结构，作为传统前缀树(Trie)的替代。与传统前缀树不同，基数树的边不仅可以用单个元素标记，还可以用不同长度的元素序列标记，大大提高了效率。

使用基数树来管理token序列及其对应的KV缓存张量的映射。这些KV缓存张量以非连续的、分页的布局存储，其中每页的大小相当于一个token。

因为GPU内存很快被KV缓存填满，引入了一个简单的LRU逐出策略，**首先逐出最近最少使用的叶子**。通过先逐出叶子，使得它们的公共祖先可以被重用，直到这些祖先变成叶子并被逐出。



SGLang系统中的RadixAttention技术

Thinking: 怎么理解基数树(Radix Tree)的数据结构细节?

传统前缀树(Trie)像逐字档案柜:

- 找"北京市朝阳区"需要经过: 北→京→市→朝→阳→区 (6个抽屉)
- 每个字都要单独开一个抽屉

基数树(Radix Tree)像智能压缩文件夹:

- 直接有一条边标记"北京市朝阳区" (作为一个整体)
- 或者标记"北京市" (共同前缀), 下面再分叉"朝阳区"/"海淀区"
- 边可以压缩长序列, 大大提高查找效率

SGLang系统中的RadixAttention技术

Thinking: 针对非连续分页存储，如何处理？

- 就像你把档案拆开存在多个抽屉，但有一张索引卡记录完整路径
- 淘汰时（比如柜子满了），你先把"朝阳区-某小区-具体门牌"这个具体档案（叶子）抽掉
- 但保留"北京市朝阳区"这个文件夹（公共祖先），因为其他人可能还要用

CASE：AI客服中心示例（RadixAttention技术）

Thinking: 针对AI客服中心，RadixAttention如何处理真实请求？

假设你是一个AI客服系统的管理员，系统要处理多个用户的对话。

步骤(1): 初始状态

基数树是空的，就像刚开业的客服中心，没有任何历史记录

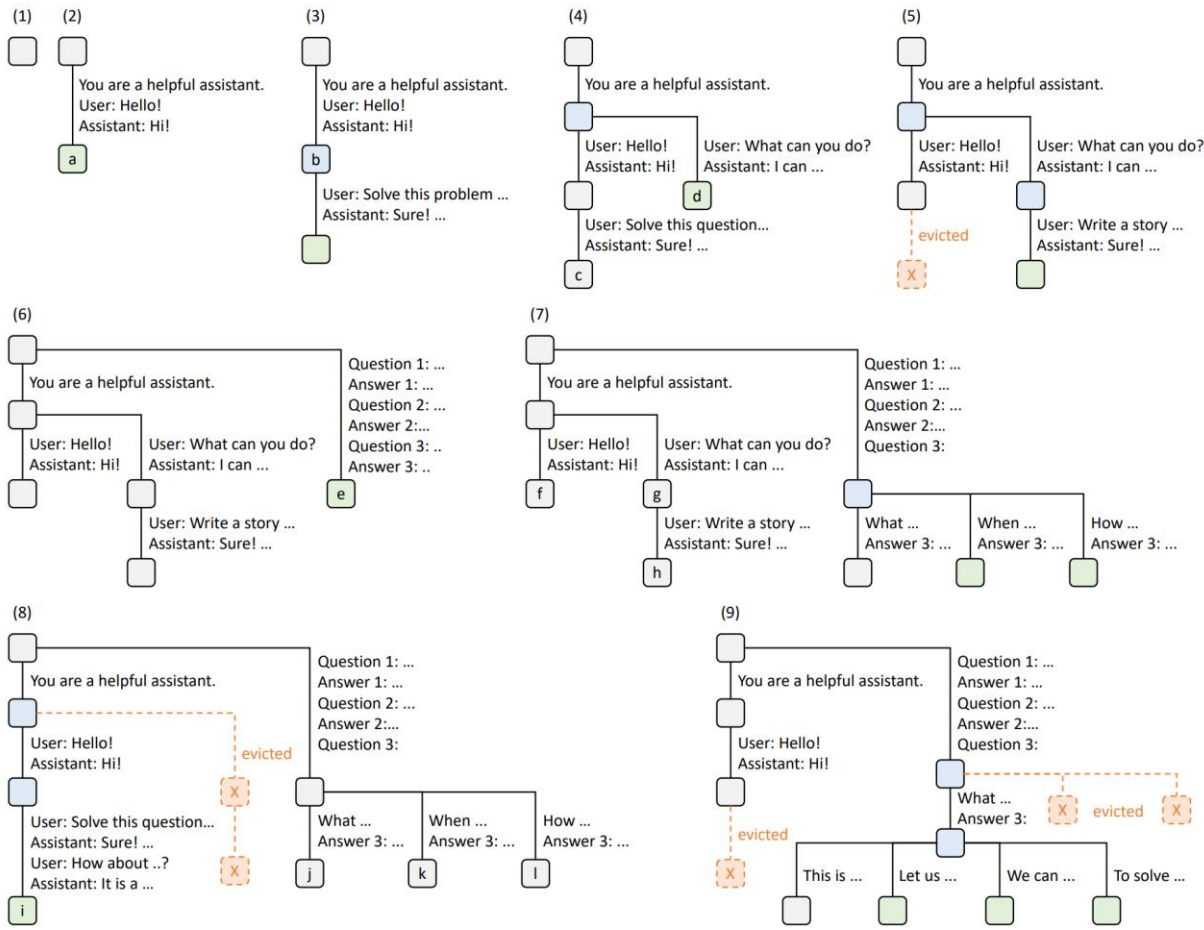
步骤(2): 第一个客户（节点a）

用户发送: "Hello"

系统提示: "你是一个有帮助的助手"

建立路径: 根节点 → ["你是一个有帮助的助手。User: Hello!
Assistant: Hi!"] → 节点a

绿色节点a: 新建的缓存



使用最近最少使用（LRU）淘汰策略的RadixAttention操作的示例，示例跨越了9个时间点，响应各种请求时基数树的动态演变。

CASE: AI客服中心示例 (RadixAttention技术)

步骤(3): 同一个客户继续对话 (节点b)

用户继续问: "Solve this problem..."

复用前缀: 系统发现"你是一个有帮助的助手。User: Hello!

Assistant: Hi!"这部分已经存在 (蓝色节点)

新增: 只计算"User: Solve this problem..."这部分, 挂在节点b下

效果: 第一轮对话的KV Cache直接复用, 不用重新计算

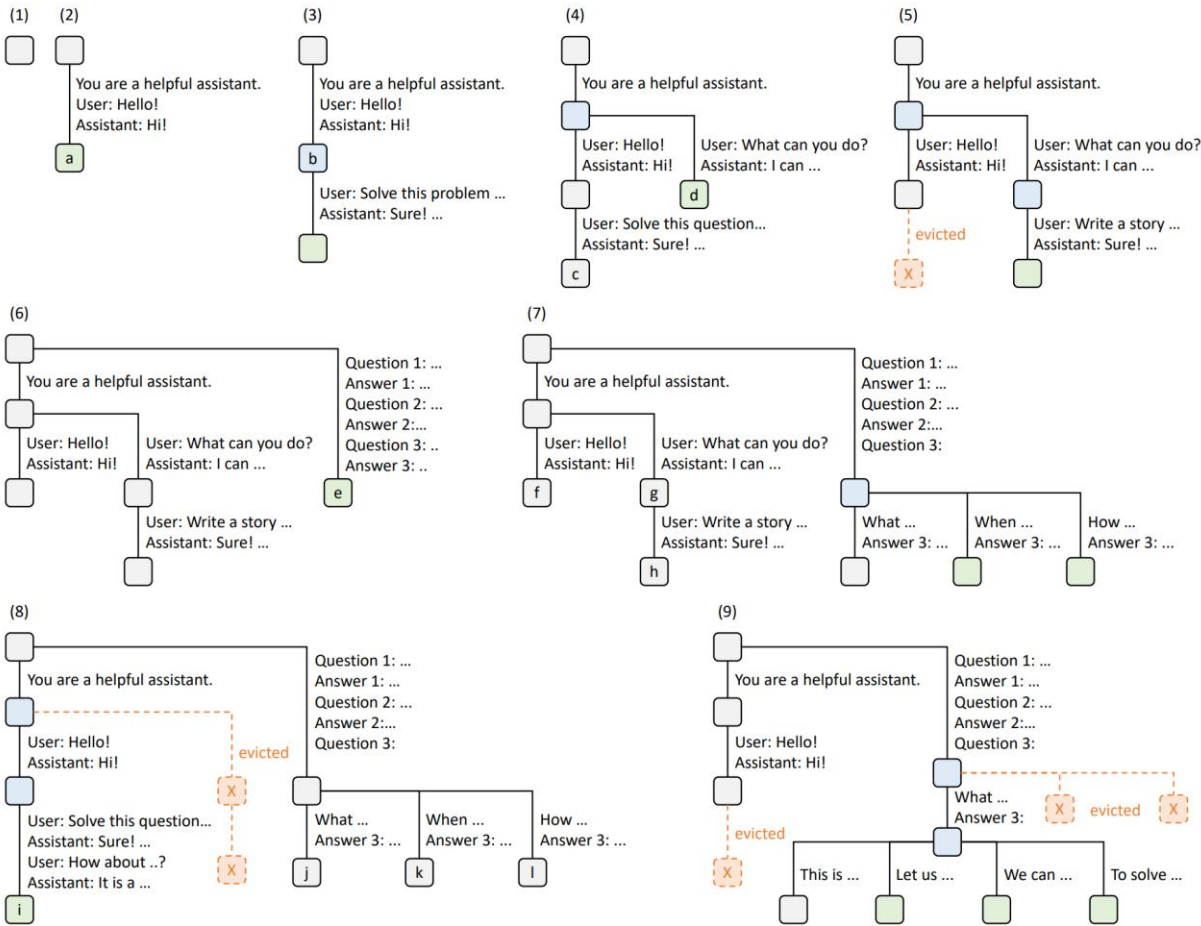
步骤(4): 第二个客户加入 (节点d)

新客户发送: "What can you do?"

共享前缀: 系统提示"你是一个有帮助的助手"是共同的

操作: 把节点b拆分, 让两个客户共享"系统提示+第一轮回合"这个前缀

分支: 现在树上有两个分支, 共享同一个根和主干



- 绿色: 刚加入的新节点 (新计算的KV Cache)
- 蓝色: 被访问到的缓存节点 (复用的KV Cache)
- 红色: 被驱逐的节点 (释放内存)

CASE: AI客服中心示例 (RadixAttention技术)

步骤(5): 内存不足, 开始淘汰 (节点c被驱逐)

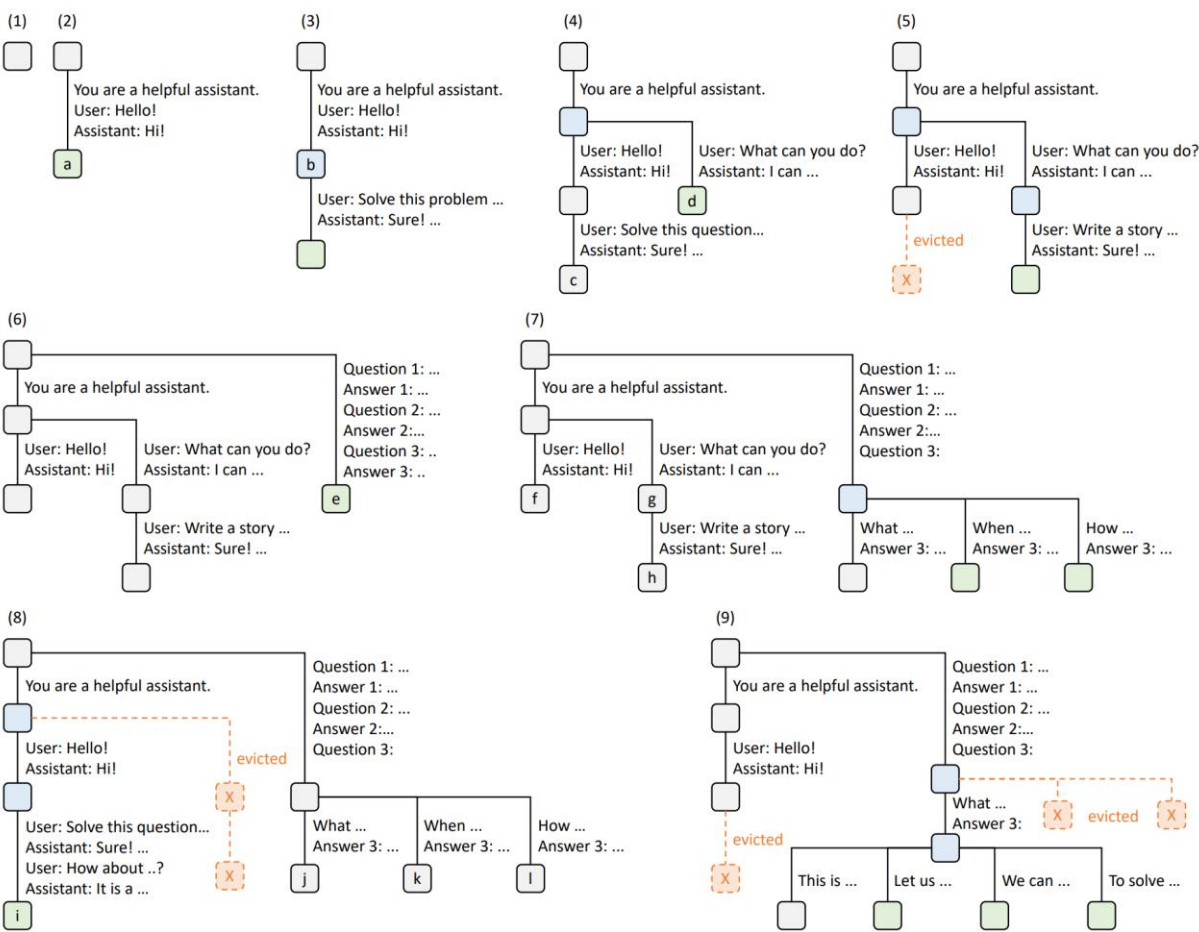
第二个客户继续问: "Write a story..."

问题: GPU显存快满了

LRU淘汰: 节点c (第一个客户的第二回合) 是最久未使用的叶子, 被标记为红色驱逐

保留: 节点d (第二个客户的第一轮) 保留, 因为刚用过

新增: 在节点d下添加"User: Write a story..."



使用最近最少使用 (LRU) 淘汰策略的RadixAttention操作的示例, 示例跨越了9个时间点, 响应各种请求时基数树的动态演变。

CASE：AI客服中心示例（RadixAttention技术）

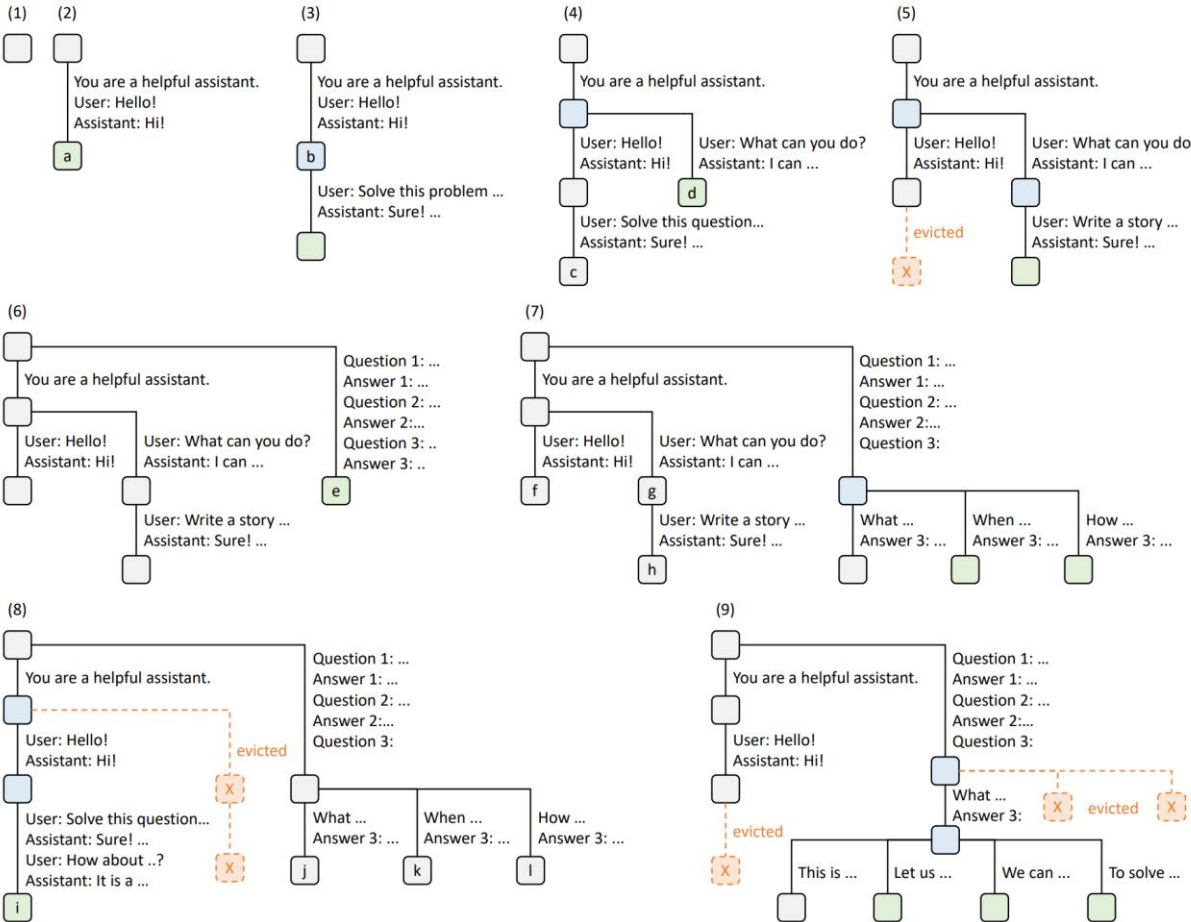
步骤(6)：批量处理相似问题（少样本学习）

来了20个用户，都问类似的数学题，带有示例"Question 1... Answer 1..."

操作：根节点被分割，因为这些新问题与之前的聊天记录没有共同前缀

新建分支：在节点e处建立"Question 1/Answer 1/Question 2/Answer 2..."的链条

优势：20个用户共享同一套"示例问答"的KV Cache



使用最近最少使用（LRU）淘汰策略的RadixAttention操作的示例，示例跨越了9个时间点，响应各种请求时基数树的动态演变。

CASE：AI客服中心示例（RadixAttention技术）

步骤(7)：更多批量查询

新来一批查询，共享同一组少样本示例

操作：分割节点e，让新旧查询共享示例部分

效果：就像把"例题讲解"这部分视频缓存共享给所有学生，只记录每个学生的"具体答案"差异

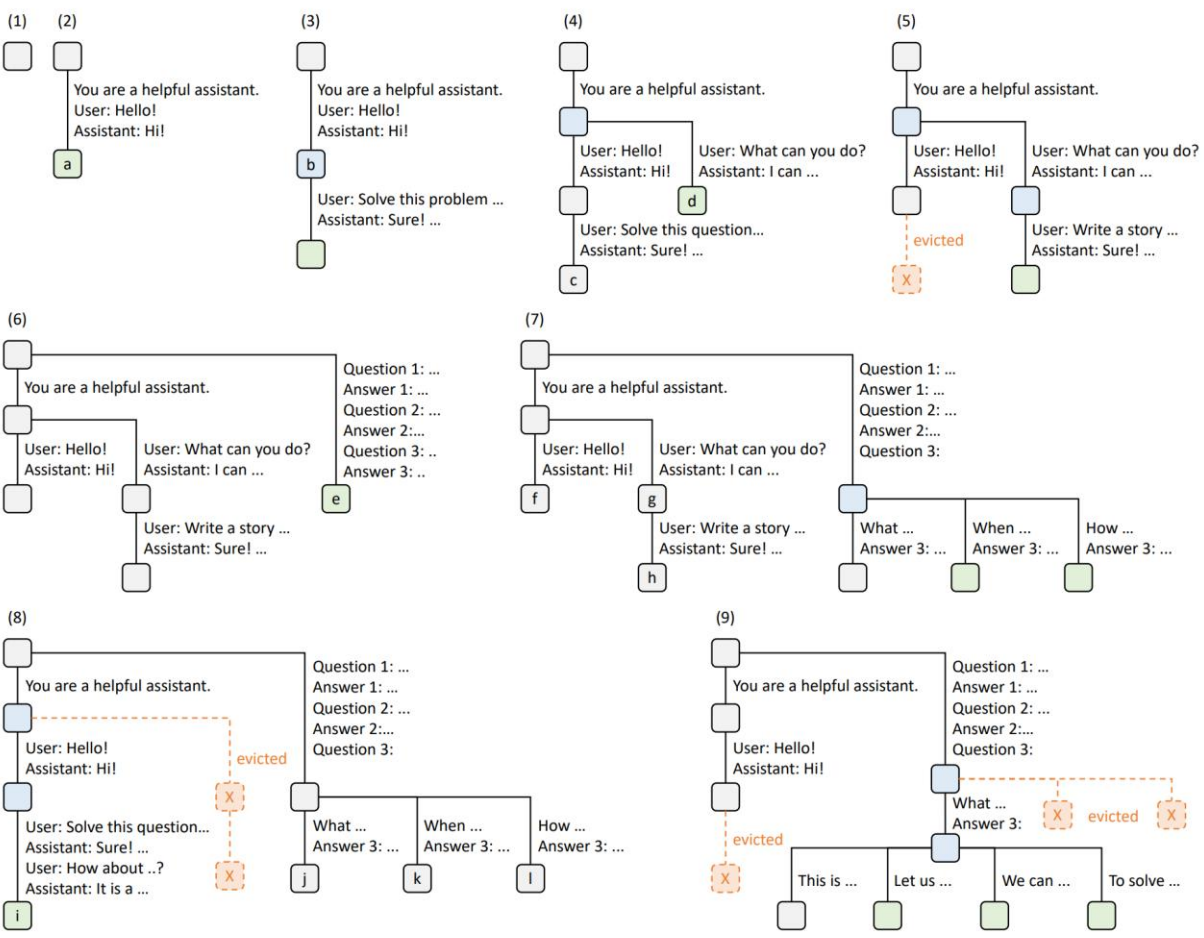
步骤(8)：回到第一个客户（驱逐第二个客户）

第一个客户（步骤3的节点b路径）发来新消息

LRU淘汰：为了腾出空间，驱逐第二个客户的所有节点（g和h），因为它们最近最少使用

保留：第一个客户的路径保留（蓝色节点）

新增：在第一个客户下添加新内容（节点i,j）



使用最近最少使用（LRU）淘汰策略的RadixAttention操作的示例，示例跨越了9个时间点，响应各种请求时基数树的动态演变。

CASE: AI客服中心示例 (RadixAttention技术)

步骤(9): 自我一致性采样 (多个答案)

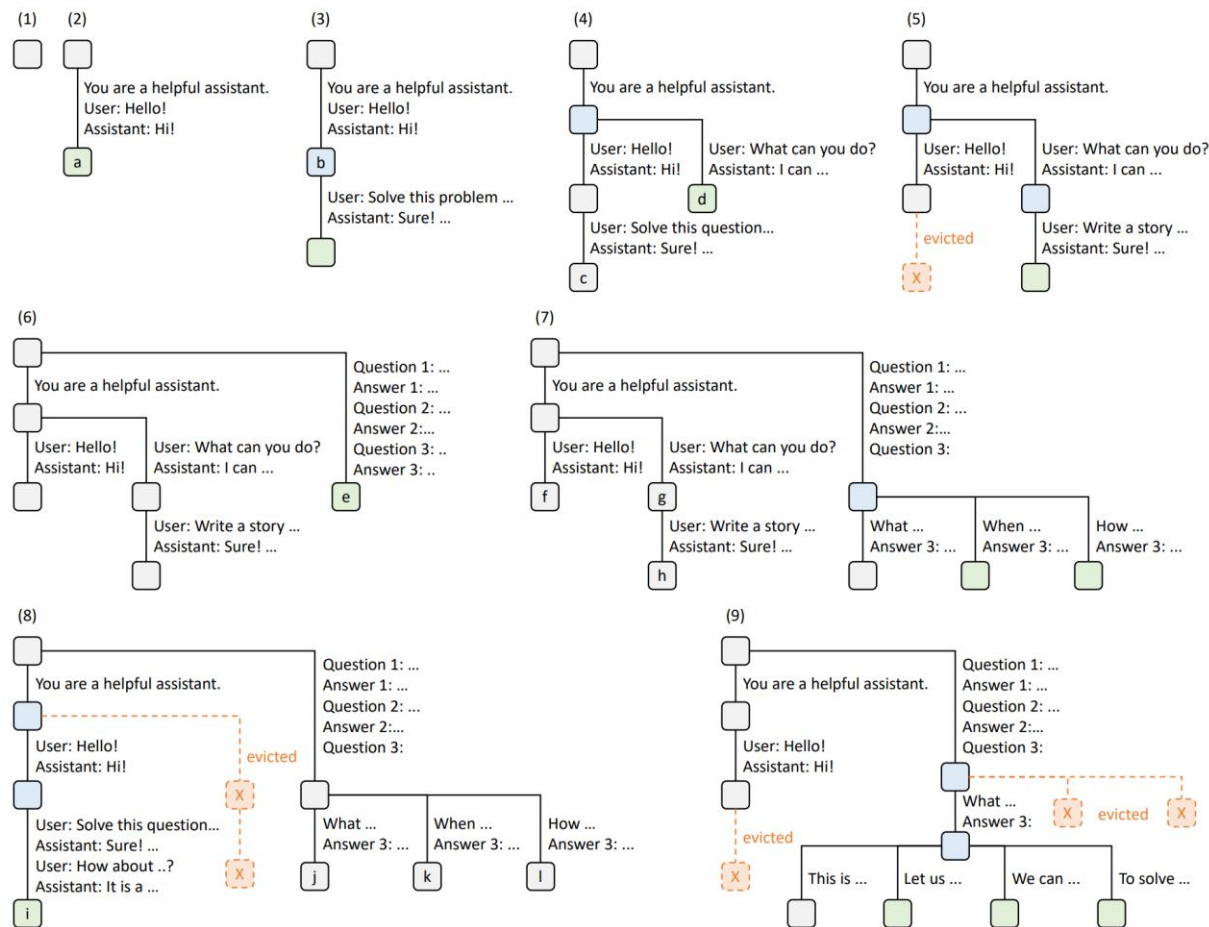
对节点j的问题，需要生成3个不同版本的答案

(What/When/How)

操作：为了腾出空间，驱逐节点i、k、l（其他分支的叶子）

新增：在节点j下分叉出4个叶子节点，保存不同答案变体

复用：Question 3的KV Cache只计算一次，4个答案共享前缀



使用最近最少使用（LRU）淘汰策略的RadixAttention操作的示例，示例跨越了9个时间点，响应各种请求时基数树的动态演变。

缓存感知调度

缓存感知调度

- 缓存命中率 = 缓存提示token的数量 / 提示token的总数。

当等待队列中有许多请求时，它们执行的顺序可以显著影响缓存命中率。

例如，如果请求调度器经常在不同的、不相关的请求之间切换，会导致缓存抖动和低命中率。

作者设计了一个缓存感知调度算法来提高缓存命中率。在批处理设置中，我们根据匹配的前缀长度对请求进行排序，并优先处理匹配前缀更长的请求，而不是使用先到先服务的时间表。

Algorithm 1 Cache-Aware Scheduling for RadixAttention with Continuous Batching.

Input: The radix tree T , the memory pool P , the current running batch B , the waiting queue Q .

Output: Finished requests and updated system state.

```
// Get all requests from the waiting queue
requests ← Q.get_all_requests()
// Search for the prefix matching for all waiting requests
for req ∈ requests do
    req.prefix_node, req.prefix_len ← T.match_prefix(req.input_tokens)
end for
// Sort the requests according to the matched prefix lengths
requests.sort()
// Select requests for the next batch
available_size ← T.evictable_size() + P.available_size()
current_size ← 0
new_batch ← []
for req ∈ requests do
    if req.size() + current_size < available_size then
        new_batch.append(req)
        delta ← T.increase_ref_counter(req.prefix_node)
        available_size ← available_size + delta
    end if
end for
Q.remove_requests(new_batch)
// Insert requests into the current running batch
B.merge(new_batch)
// Allocate new memory and do eviction if necessary
needed_size ← B.needed_size()
success, buffer ← P.alloc(needed_size)
if not success then
    T.evict(needed_size)
    success, buffer ← P.alloc(needed_size)
end if
B.run(buffer)
// Process finished requests
finished_requests ← B.drop_finished_requests()
for req ∈ finished_requests do
    T.decrease_ref_counter(req.prefix_node)
    T.insert(req)
end for
return finished_requests
```

缓存感知调度

算法1显示了连续批处理的缓存感知调度的伪代码。

输入：基数树T，内存池P，当前运行的批次B，等待队列Q。

输出：完成的请求和更新后的系统状态。

算法步骤

1、获取等待队列中的所有请求：将等待队列中的所有请求取出。

2、为所有等待的请求匹配前缀：对于等待队列中的每个请求，使用基数树T来匹配请求输入token的前缀。（这一步骤是确定哪些请求可以重用已有的KV缓存）

3、根据匹配的前缀长度对请求排序：将请求根据匹配到的前缀长度进行排序，优先处理那些匹配前缀更长的请求。

4、选择下一批处理的请求：

计算出当前可分配的总内存大小（包括可逐出的缓存大小和内存池的可用大小）。

遍历排序后的请求，选择那些可以被当前内存大小处理的请求组成新的批次。

Algorithm 1 Cache-Aware Scheduling for RadixAttention with Continuous Batching.

Input: The radix tree T , the memory pool P , the current running batch B , the waiting queue Q .

Output: Finished requests and updated system state.

```
// Get all requests from the waiting queue
requests ← Q.get_all_requests()
// Search for the prefix matching for all waiting requests
for req ∈ requests do
    req.prefix_node, req.prefix_len ← T.match_prefix(req.input_tokens)
end for
// Sort the requests according to the matched prefix lengths
requests.sort()
// Select requests for the next batch
available_size ← T.evictable_size() + P.available_size()
current_size ← 0
new_batch ← []
for req ∈ requests do
    if req.size() + current_size < available_size then
        new_batch.append(req)
        delta ← T.increase_ref_counter(req.prefix_node)
        available_size ← available_size + delta
    end if
end for
Q.remove_requests(new_batch)
// Insert requests into the current running batch
B.merge(new_batch)
// Allocate new memory and do eviction if necessary
needed_size ← B.needed_size()
success, buffer ← P.alloc(needed_size)
if not success then
    T.evict(needed_size)
    success, buffer ← P.alloc(needed_size)
end if
B.run(buffer)
// Process finished requests
finished_requests ← B.drop_finished_requests()
for req ∈ finished_requests do
    T.decrease_ref_counter(req.prefix_node)
    T.insert(req)
end for
return finished_requests
```

缓存感知调度

5、从等待队列中移除选中的请求：将选定的请求从等待队列中移除，准备将它们加入当前运行的批次。

6、将请求插入当前运行的批次：将选中的请求合并到当前运行的批次中。

7、分配新内存并进行逐出（如果必要的话）：

根据当前运行批次的需求计算需要分配的内存大小。

如果内存分配成功，则继续执行；如果不成功，则从基数树中逐出一部分缓存以释放内存。

8、执行当前运行的批次中的所有请求

9、处理完成的请求：

从当前运行的批次中提取已完成的请求。

对于每个完成的请求，更新基数树中的引用计数器，并将请求插入到基数树中以供后续使用。

10、返回完成的请求：返回所有已完成的请求，这些请求的执行结果可以被用于进一步的处理或直接输出。

Algorithm 1 Cache-Aware Scheduling for RadixAttention with Continuous Batching.

Input: The radix tree T , the memory pool P , the current running batch B , the waiting queue Q .

Output: Finished requests and updated system state.

```
// Get all requests from the waiting queue
requests ← Q.get_all_requests()
// Search for the prefix matching for all waiting requests
for req ∈ requests do
    req.prefix_node, req.prefix_len ← T.match_prefix(req.input_tokens)
end for
// Sort the requests according to the matched prefix lengths
requests.sort()
// Select requests for the next batch
available_size ← T.evictable_size() + P.available_size()
current_size ← 0
new_batch ← []
for req ∈ requests do
    if req.size() + current_size < available_size then
        new_batch.append(req)
        delta ← T.increase_ref_counter(req.prefix_node)
        available_size ← available_size + delta
    end if
end for
Q.remove_requests(new_batch)
// Insert requests into the current running batch
B.merge(new_batch)
// Allocate new memory and do eviction if necessary
needed_size ← B.needed_size()
success, buffer ← P.alloc(needed_size)
if not success then
    T.evict(needed_size)
    success, buffer ← P.alloc(needed_size)
end if
B.run(buffer)
// Process finished requests
finished_requests ← B.drop_finished_requests()
for req ∈ finished_requests do
    T.decrease_ref_counter(req.prefix_node)
    T.insert(req)
end for
return finished_requests
```

Summay (缓存感知调度)

缓存命中率 = 缓存命中的token数 / 总token数

问题：如果随机调度请求，会导致缓存"抖动"（刚加载的缓存马上被换出）

解决方案：优先处理前缀匹配最长的请求，而不是先来先服务(FCFS)

比如：机场安检智能排队

传统方式（先到先服务）：

- 乘客A去东京，乘客B去纽约，乘客C去东京，乘客D去伦敦
- 按顺序：A(东京)→B(纽约，换设备)→C(东京，重新准备)→D(伦敦，换设备)
- 问题：东京的安检设备刚准备好，B来了要换成纽约的，C来了又要换回东京的——频繁切换浪费资源

缓存感知调度（SGLang方式）：

- 系统看到等待队列中有A(东京)、C(东京)、E(东京)、B(纽约)、D(伦敦)
- 排序：让A、C、E（都是东京）优先连续处理
- 效果：东京的安检设备状态（KV Cache）保持不动，连续服务3个人，大幅提升效率

Summary (RadixAttention技术)

- 基数树 (Radix Tree) :

RadixAttention使用基数树来管理token序列和对应的KV缓存张量之间的映射。

基数树允许快速的前缀搜索、重用、插入和逐出操作。

- LRU (最近最少使用) 淘汰策略:

为了有效管理内存，实现了LRU缓存淘汰策略。

- 缓存感知调度

通过缓存感知的调度策略，优先处理那些最有可能重用现有KV缓存的请求，提高处理效率和吞吐量。

通过动态地管理内存和缓存，算法能够适应不同负载下的需求，保持高效的运行状态。

RadixAttention技术是由LMSYS组织在它的SGLang项目中提出的

根据LMSYS组织的研究，RadixAttention可以显著提高LLM效率。在A10G GPU上使用Llama-7B和Mixtral-8x7B模型进行的测试表明，与现有系统（如Guidance和vLLM）相比，采用RadixAttention的SGLang系统实现了高达5倍的吞吐量提升

约束解码的需求

- 约束解码的需求

在LLM应用中，经常需要将模型的输出限制为符合特定格式，例如JSON。这种需求可以使用正则表达式，让输出的控制性和鲁棒性增强。

- 现有系统的局限性

传统的解码方法在处理约束时，通常一次只解码一个token，这在有多个token可以同时满足约束条件时效率较低。这种方法在处理长序列或复杂约束时尤其低效。

- 压缩有限状态机（Compressed FSM）

为了解决这个问题，SGLang引入了压缩有限状态机，可以在解码过程中同时处理多个token。

这种方法通过构建一个状态机来表示解码过程中可能的状态转换，并在可能的情况下将多个token的解码合并到一个步骤中。

高效约束解码 (Compressed FSM)

SGLang提供了一个正则表达式参数，使用正则表达式来强制执行这些约束，这些表达式对于许多实际场景来说已经够用。系统通过将正则表达式转换为有限状态机 (FSM)。

在解码过程中，维护当前的FSM状态，从下一个状态检索允许的标记，并将无效标记的概率设置为零，逐个标记进行解码。

然而，当有机会一次性解码多个标记时，这种逐个标记的方法效率低下。例如，图中的常量序列{"summary": "在正常解码过程中跨越了多个标记

```
dimensions = ["Clarity", "Originality", "Evidence"]

@function
def multi_dimensional_judge(s, path, essay):
    s += system("Evaluate an essay about an image.")
    s += user(image(path) + "Essay:" + essay)
    s += assistant("Sure!")

    # Return directly if it is not related
    s += user("Is the essay related to the image?")
    s += assistant(select("related", choices=["yes", "no"]))
    if s["related"] == "no": return

    # Judge multiple dimensions in parallel
    forks = s.fork(len(dimensions))
    for f, dim in zip(forks, dimensions):
        f += user("Evaluate based on the following dimension:" +
            dim + ". End your judgment with the word 'END'")
        f += assistant("Judgment:" + gen("judgment", stop="END"))

    # Merge the judgments
    judgment = "\n".join(f["judgment"] for f in forks)

    # Generate a summary and a grade. Return in the JSON format.
    s += user("Provide the judgment, summary, and a letter grade")
    s += assistant(judgment + "In summary," + gen("summary", stop=".")
        + "The grade of it is" + gen("grade"))

    schema = r'{"summary": "[\w\d\s]+\. ", "grade": "[ABCD][+-]? "\}'
    s += user("Return in the JSON format.")
    s += assistant(gen("output", regex=schema))

state = multi_dimensional_judge.run(...)
print(state["output"])
```

Handle chat template and multi-modal inputs

Select an option with the highest probability

Fetch result; Use Python control flow

Runtime optimization: KV Cache Reuse (Sec. 3)

Multiple generation calls run in parallel

Fetch generation results

Runtime optimization: API speculative execution (Sec. 5)

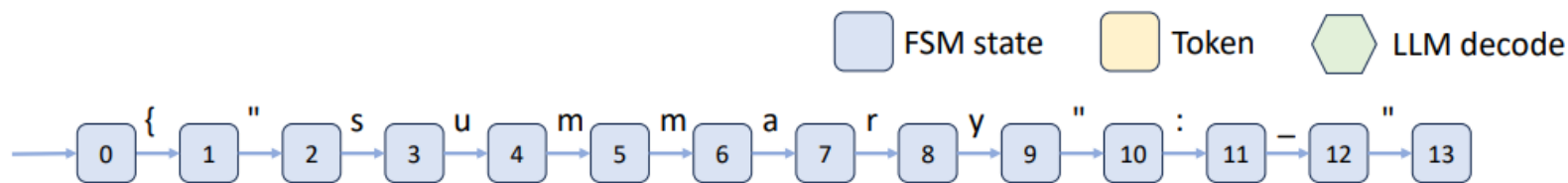
Runtime optimization: fast constrained decoding (Sec. 4)

Run an SGLang program

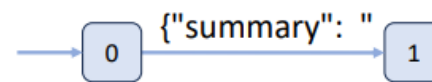
高效约束解码 (Compressed FSM)

如图 (c) 所示, 需要多个解码阶段, 即使在解码时只有一个有效的下一个标记。因为现有系统中FSM和模型运行器之间缺乏集成, 阻止了多标记处理, 导致解码速度慢。

SGLang通过创建一个带有压缩FSM的快速约束解码运行时来克服这个限制。分析FSM并将FSM中相邻的单次转换边压缩成单个边, 如图 (b) 所示, 允许它识别何时可以一起解码多个标记。在图 (d) 中, 压缩转换边上的多个标记可以在一个前向传递中解码, 这大大加速了解码过程。它也是通用的, 适用于所有正则表达式。



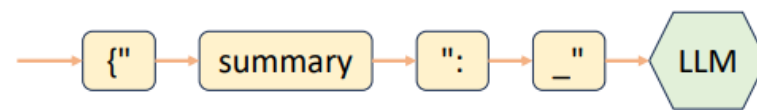
(a) Normal FSM for regex {"summary": "



(b) Compressed FSM for regex {"summary": "



(c) Decoding process with normal FSM



(d) Decoding process with compressed FSM

正常和压缩的有限状态机 (FSM) 的解码过程 (下划线_表示空格)

通过使用Compressed FSM, SGLang能够减少解码步骤, 加快了解码速度。这种方法特别适用于长序列和复杂约束的解码任务。

API推测执行

- 通过API推测执行，来提高只能通过API接口访问的大型语言模型（如OpenAI的GPT-4）的端点调用效率

尤其是商业模型API（如GPT-4），只能通过黑盒API接口访问 => 无法直接修改模型内部。

在这些API模型中，每次生成（gen）操作需要一次API调用，涉及计算和经济成本。

API推测执行

- API推测执行是一种技术，它允许系统在进行实际API调用之前，预测并生成可能需要的数据。这种方法通过并行生成多个可能的输出来减少必要的API调用次数。
- 在SGLang中，可以通过在第一次API调用时生成多个token，而不是仅仅生成一个token来实现。这些额外生成的token可以被缓存，并用于后续的生成任务，从而减少了对API的重复调用。

API推测执行

API推测执行示例（角色描述生成）：

- 假设有一个程序需要生成一个角色的名称和职业描述。
在传统的API调用中，可能需要两次调用：一次生成名称，一次生成职业。

```
s += context + "name:" + gen("name", stop="\n") + "job:" +  
gen("job", stop="\n")。
```

简单地说，这两个gen原语对应于两个API调用，这意味着用户需要为上下文token的输入费支付两次。

在SGLang中，可以在第一个调用上启用推测执行，并让它通过忽略停止条件来继续生成一些额外的token。解释器保留额外生成的输出，并与后续原语匹配和重用。

API推测执行优势

- 减少延迟：减少API调用次数，减少总响应时间。
- 成本效益：减少调用次数直接降低了成本。
- 提高效率：这种方法允许更有效地利用计算资源，因为它通过并行处理减少了等待时间。

API推测执行

Thinking: API推测的挑战是什么？

- 预测准确性：

⇒推测执行的成功依赖于预测的准确性。

如果预测的token与实际需要的token不匹配，可能会导致额外的处理成本。

- 资源管理：

⇒ 需要有效的机制来管理和缓存生成的token，以确保它们可以被正确地重用。

API推测执行通过智能地预测和生成数据，优化了对黑盒API模型的调用过程。
这不仅提高了性能，还降低了成本。

Summay (API推测执行)

API推测执行 (API Speculative Execution) 是 SGLang 针对商业 API 模型 (如 GPT、Claude 等) 设计的一种省钱+提速策略。

Thinking: 什么时候考虑用 API 推测执行?

- 需要使用 GPT/Claude 等商业 API
- 程序结构很规律 (如固定提取 3 个字段: name/job/age)
- 能接受 10-20% 的 token 浪费率换取延迟降低

Thinking: 什么时候不要用?

使用本地开源模型 (用 RadixAttention 更香)

程序有复杂分支 (if/else 逻辑), 难以预测下一步

大多数生产环境要么全用本地模型 (享受 RadixAttention 的免费缓存), 要么直接调 OpenAI API 用它官方的 Prompt Caching 功能

实验设置

实验目的

- SGLang是用PyTorch实现的，测试的目的是验证SGLang在实际应用中的表现，包括吞吐量（每秒处理的请求数量）和延迟（处理单个请求所需的时间）

实验设置：

- 模型：使用不同大小的模型，包括Llama-2、Mistral、LLaVA图像和视频模型，以及OpenAI的GPT-3.5 API模型。模型的参数从7亿到70亿不等，使用了float16精度。
- 硬件：在配备NVIDIA A10G GPU的AWS EC2 G5实例上进行，对于更大的模型，使用了多个A10G GPU进行并行处理，部分实验也在A100G GPU上进行。

• 基线比较：

SGLang与现有的高级编程系统和低级推理引擎进行比较，这些系统和引擎具有不同的语言和默认运行时环境。

实验结果

端到端性能 (End-to-End Performance)

- 开源模型的结果：在Llama-7B模型上，SGLang能够显著提高吞吐量（高达6.4倍）并减少延迟（高达3.7倍）。这些归因于KVCache重用、程序内并行性和更快的约束解码。
- 不同基准测试的加速原因：例如，在MMLU基准测试中，SGLang通过重用KV缓存来提高性能。在HellaSwag基准测试中，SGLang利用few-shot示例和问题前缀的KV缓存来提高性能。

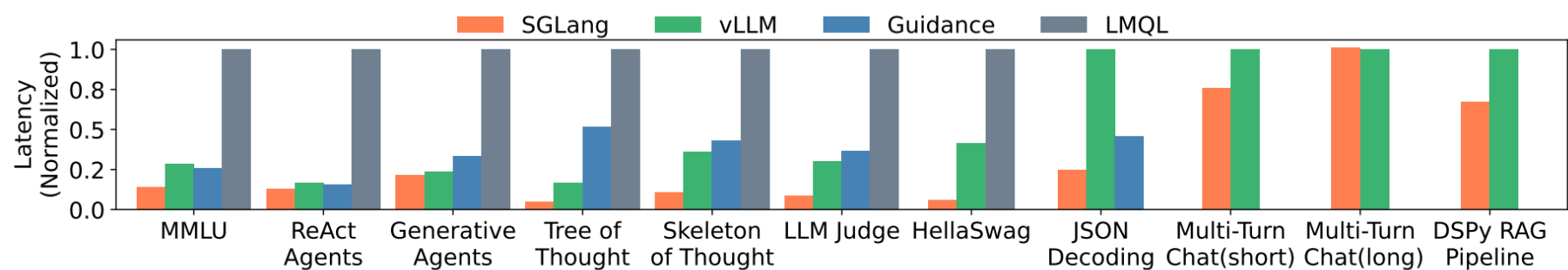


Figure 6: Normalized latency on Llama-7B models. Lower is better.

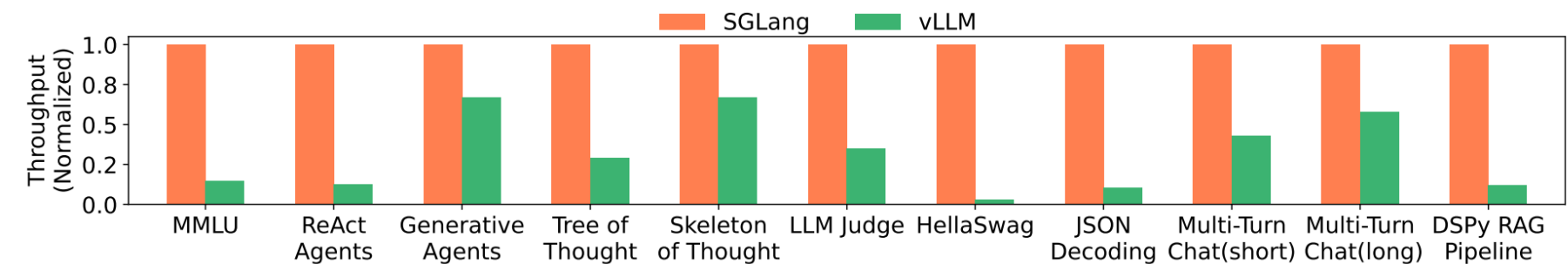


Figure 7: Normalized throughput on Mixtral-8x7B models with tensor parallelism. Higher is better.

SGLang使用

1. 环境准备（必须新建环境）

SGLang 与 vLLM 依赖冲突（sgl-kernel、triton 等），必须隔离：

```
# 在数据盘创建独立环境
cd /root/autodl-tmp
python -m venv sglang_env
source /root/autodl-tmp/sglang_env/bin/activate

# 设置 pip 缓存到数据盘（防止系统盘满）
export PIP_CACHE_DIR=/root/autodl-tmp/pip-cache
```

SGLang 依赖的 FlashInfer、sgl-kernel 等组件要求 CUDA 12.4。如果你的 AutoDL 实例是 CUDA 11.8，启动时会报各种底层错误。

在 AutoDL 创建实例时，请选择 CUDA 12.4 的镜像（如 PyTorch 2.5.1 + CUDA 12.4）。如果当前实例是 11.8，需要关机后更换镜像

SGLang使用

2. 安装依赖（顺序很重要）

Step 1: 升级 pip

```
pip install --upgrade pip
```

Step 2: 安装 sgl-kernel（无依赖模式，避免冲突）

```
pip install sgl-kernel --force-reinstall --no-deps
```

Step 3: 安装 SGLang 和 flashinfer（指定 torch2.5 版本）

```
pip install "sglang[all]>=0.4.6.post1" --find-links
```

```
https://flashinfer.ai/whl/cu124/torch2.5/flashinfer-python
```

Step 4: 安装指定 transformers（5.x 会报错）

```
pip install transformers==4.57.1
```

SGLang使用

3. 启动服务

设置 CUDA 库路径 (解决 libnvrtc.so.12 错误)

```
export LD_LIBRARY_PATH=/root/autodl-
```

```
tmp/sglang env/lib/python3.10/site-
```

```
packages/nvidia/cuda nvidia/lib:$LD_LIBRARY_PATH
```

启动 (0.6B 小模型无需张量并行)

```
python -m sglang.launch server \
```

```
--model-path /root/autodl-tmp/models/qwen/Qwen3-0.6B \
```

```
--port 8001 \
```

```
--host 0.0.0.0 \
```

```
--mem-fraction-static 0.6
```

```
[2026-03-23 13:08:33] Load weight begin. avail mem=23.06 GB
Loading safetensors checkpoint shards: 0% Completed | 0/1 [00:00<?, ?it/s]
[2026-03-23 13:08:33] Parameter lm_head.weight not found in params dict
Loading safetensors checkpoint shards: 100% Completed | 1/1 [00:00<00:00, 1.87it/s]
Loading safetensors checkpoint shards: 100% Completed | 1/1 [00:00<00:00, 1.87it/s]

[2026-03-23 13:08:34] Load weight end. elapsed=0.64 s, type=Qwen3ForCausalLM, dtype=torch.bfloat16, avail mem=21.88 GB, mem usage=1.18 GB.
[2026-03-23 13:08:34] Using KV cache dtype: torch.bfloat16
[2026-03-23 13:08:34] KV Cache is allocated. #tokens: 118372, K size: 6.32 GB, V size: 6.32 GB
[2026-03-23 13:08:34] Memory pool end. avail mem=8.87 GB
[2026-03-23 13:08:34] Capture cuda graph begin. This can take up to several minutes. avail mem=8.30 GB
[2026-03-23 13:08:34] Capture cuda graph bs [1, 2, 4, 8, 12, 16, 24]
Capturing batches (bs=1 avail mem=8.15 GB): 100%
[2026-03-23 13:09:18] Capture cuda graph end. Time elapsed: 44.08 s. mem usage=0.15 GB, avail mem=8.15 GB.
[2026-03-23 13:09:19] max_total_num_tokens=118372, chunked_prefill_size=2048, max_prefill_tokens=16384, max_running_requests=2048, context_len=4096,
able gpu mem=8.15 GB
[2026-03-23 13:09:19] INFO: Started server process [23101]
[2026-03-23 13:09:19] INFO: Waiting for application startup.
[2026-03-23 13:09:19] Using default chat sampling params from model generation config: {'repetition_penalty': 1.0, 'temperature': 0.6, 'top_k':
: 0.95}
[2026-03-23 13:09:19] INFO: Application startup complete.
[2026-03-23 13:09:19] INFO: Uvicorn running on http://0.0.0.0:8001 (Press CTRL+C to quit)
[2026-03-23 13:09:20] INFO: 127.0.0.1:33320 - "GET /model_info HTTP/1.1" 200 OK
[2026-03-23 13:09:36] Prefill batch, #new-seq: 1, #new-token: 6, #cached-token: 0, token usage: 0.00, #running-req: 0, #queue-req: 0, input thro
en/s): 0.00, cuda graph: False
[2026-03-23 13:09:36] INFO: 127.0.0.1:33332 - "POST /generate HTTP/1.1" 200 OK
[2026-03-23 13:09:36] INFO: The server is fired up and ready to roll!
[2026-03-23 13:09:44] Prefill batch, #new-seq: 1, #new-token: 1, #cached-token: 0, token usage: 0.00, #running-req: 0, #queue-req: 0, input thro
en/s): 0.75, cuda graph: False
[2026-03-23 13:09:45] INFO: 127.0.0.1:54520 - "GET /health HTTP/1.1" 200 OK
/root/autodl-tmp/sglang_env/lib/python3.12/site-packages/fastapi/routing.py:116: FastAPIDeprecationWarning: ORJSONResponse is deprecated. FastAP
lizes data directly to JSON bytes via Pydantic when a return type or response model is set, which is faster and doesn't need a custom response c
more in the FastAPI docs: https://fastapi.tiangolo.com/advanced/custom-response/#orjson-or-response-model and https://fastapi.tiangolo.com/tutor
e-model/
response = await f(request)
[2026-03-23 13:09:55] INFO: 127.0.0.1:36976 - "GET /v1/models HTTP/1.1" 200 OK
```

SGLang使用

RTX 4090 推荐配置 (Qwen3-8B)

RTX 4090 (24GB) 跑 Qwen3-8B 的推荐配置

```
python -m sglang.launch_server \  
  --model-path /root/autodl-tmp/models/qwen/Qwen3-8B \  
  --port 8001 \  
  --host 0.0.0.0 \  
  --mem-fraction-static 0.85 \      # 显存利用率 85% (4090 留一些给系统)  
  --max-running-requests 16 \      # 最大并发请求数  
  --context-length 4096 \          # 上下文长度 (越短越省显存)  
  --chunked-prefill-size 2048 \    # 分块预填充 (降低 TTFT)  
  --enable-metrics                 # 开启监控指标
```

SGLang使用

验证服务是否启动成功

等待启动完成（通常 30 秒 ~ 2 分钟），看到 "Running on http://0.0.0.0:8001" 即成功

健康检查

```
curl http://localhost:8001/health
```

查看模型信息

```
curl http://localhost:8001/v1/models
```

```
(sglang_env) root@autodl-container-71d14fa7dd-aa0f4e25:~/autodl-tmp/sglang_cases# curl http://localhost:8001/v1/models
{"object": "list", "data": [{"id": "/root/autodl-tmp/models/qwen/Qwen3-0.6B", "object": "model", "created": 1774243272, "owned_by": "sglang", "root": "/root/autodl-tmp/models/qwen/Qwen3-0.6B", "parent": null, "max_model_len": 40960}]} (sglang_env) root@autodl-container-71d14fa7dd-aa0f4e25:~/autodl-tmp/sglang_cases#
```

常见启动参数

参数	说明
--port 8001	API 服务端口号
--host 0.0.0.0	允许外部访问（AutoDL 必需，否则只能 localhost 访问）
--mem-fraction-static 0.6	GPU 显存静态占用比例（0.6 = 60%），小模型设低一点，大模型可以设到 0.85
--tp-size 2	张量并行（多 GPU 时使用，单卡不需要）
--context-length 4096	最大上下文长度（越短越省显存）
--chunked-prefill-size 2048	分块预填充大小（降低首 Token 延迟）
--max-running-requests 16	最大并发请求数
--enable-metrics	开启 Prometheus 格式监控指标
--chat-template qwen3	强制指定对话模板（通常自动识别，报错时手动指定）

常见问题排查

问题 1（最常见）：CUDA 版本不对，各种底层报错

解决方案：确认 AutoDL 实例使用的是 CUDA 12.4 镜像

```
# 检查当前 CUDA 版本
```

```
nvcc --version
```

```
nvidia-smi
```

```
# 如果显示 CUDA 11.8 --> 需要关机，在 AutoDL 控制台切换为 CUDA 12.4 镜像
```

```
# 如果显示 CUDA 12.4 --> 没问题，继续
```

这是最常踩的坑。CUDA 11.8 环境下安装 SGLang, FlashInfer 和 sgl-kernel 都会报错。不要试图在 11.8 上安装，直接换镜像是最快的解决办法。

常见问题排查

问题 2: PyTorch 的 CUDA 版本与系统不匹配

解决方案: 手动指定 cu124 的 PyTorch wheel

```
# 检查当前 PyTorch 使用的 CUDA 版本
```

```
python -c "import torch; print(torch.version.cuda)"
```

```
# 如果输出不是 12.4, 重新安装正确版本
```

```
pip install torch==2.5.1 torchvision==0.20.1 torchaudio==2.5.1 \
--index-url https://download.pytorch.org/whl/cu124 --force-reinstall
```

常见问题排查

问题 3：依赖冲突（在 vLLM 环境中装了 SGLang）

解决方案：删除旧环境，重新创建纯净环境

```
# 退出当前环境
```

```
deactivate
```

```
# 删除旧环境，从头再来
```

```
rm -rf /root/autodl-tmp/sglang_env
```

```
# 重新按 Step 1 ~ Step 3 操作
```

常见问题排查

问题 4: CUDA Out of Memory (显存不足)

解决方案:

- 降低 `--mem-fraction-static` 到 0.5 ~ 0.6
- 减小 `--context-length` (如从 8192 降到 2048)
- 换用更小的模型 (Qwen3-0.6B / Qwen3-1.7B) , 先调通流程再上大模型
- 使用量化: `--quantization awq` (需要对应的 AWQ 量化模型)

SGLang 场景1：基础对话

场景1：基础对话（OpenAI 兼容 API）

最简单的用法，和调用 OpenAI API 完全一样。如果你之前的代码是调 OpenAI/vLLM 的，只需要改一下 base_url 就能切到 SGLang。

```
from openai import OpenAI

# 只需要改 base_url，其他代码完全不变
client = OpenAI(
    base_url="http://localhost:8001/v1",
    api_key="not-needed" # 本地部署不需要 key
)
response = client.chat.completions.create(
    model="Qwen3-0.6B",
    messages=[
        {"role": "system", "content": "你是一个有帮助的AI助手。"},
        {"role": "user", "content": "用一句话解释什么是大语言模型。"}
    ]
)
```

```
temperature=0.7,
max_tokens=200
)
print(response.choices[0].message.content)
```

<think>

嗯，用户让我用一句话解释什么是大语言模型。首先，我需要确定他们需要的是一个简洁明了的定义... </think>

大语言模型是一种能够理解和生成大量文本的智能系统，通过深度学习技术学习大量文本数据，具备多任务处理能力和语言理解能力。

SGLang 场景2：流式输出

场景2：流式输出（实时打字效果）

用户看到的是"一个字一个字蹦出来"的效果，体验更好。适合聊天机器人、实时翻译等场景。

```
from openai import OpenAI

client = OpenAI(base_url="http://localhost:8001/v1", api_key="not-needed")

# 流式输出：stream=True
stream = client.chat.completions.create(
    model="Qwen3-0.6B",
    messages=[
        {"role": "user", "content": "写一首关于春天的五言绝句。"}
    ],
    stream=True,
    max_tokens=200
)
```

```
# 逐块接收并打印
for chunk in stream:
    if chunk.choices[0].delta.content:
        print(chunk.choices[0].delta.content, end="", flush=True)
print() # 换行
```

<think>

好的，用户让我写一首关于春天的五言绝句。首先，我需要确认五言绝句的基本结构。五言绝句每句五个字，四句组成，通常押韵。春天的主题有很多元素，比如花开、鸟鸣、绿草、燕子等。

接下来，我得考虑如何将元素自然地融合进诗句中。可能需要用一些动词和形容词来描绘春天的景象。比如，第一句可以描述花开，第二句鸟鸣，第三句绿草，第四句夕阳或风。

2-流式输出.py

...

SGLang 场景3：多轮对话

场景3：多轮对话（利用 RadixAttention 缓存加速）

多轮对话是 SGLang 的优势场景 -- RadixAttention 会自动缓存之前轮次的 KV Cache，后续轮次直接复用。

```
from openai import OpenAI

client = OpenAI(base_url="http://localhost:8001/v1", api_key="not-needed")

# 维护对话历史
messages = [
    {"role": "system", "content": "你是一个电商客服，帮助用户查询订单和处理退款。"}
]

# 模拟多轮对话
user_inputs = [
    "我的订单 2024001 到哪了？ ",
    "那这个订单可以退款吗？ ",      # 第二轮：复用第一轮 KV Cache
    "退款需要多久到账？ ",          # 第三轮：复用前两轮 KV Cache
```

```
for user_input in user_inputs:
    messages.append({"role": "user", "content": user_input})
    response = client.chat.completions.create(
        model="Qwen3-0.6B",
        messages=messages,
        max_tokens=300,
        temperature=0.7
    )
    reply = response.choices[0].message.content
    messages.append({"role": "assistant", "content": reply})
    print(f"用户: {user_input}")
    print(f"客服: {reply}")
    print("---")
```

SGLang 场景3：多轮对话

用户: 我的订单 2024001 到哪了?

客服: <think>

好的, 用户问我的订单2024001到哪了。首先, 我需要确认用户的问题是否相关。用户可能想知道订单是否已经发货, 或者是否有物流信息。用户可能没有及时查看订单状态, 或者担心订单是否存在问题。

接下来, 我需要考虑如何回应。作为电商客服, 我应该首先询问用户是否需要帮助查询订单状态, 或者是否有其他问题需要解决。同时, 要确保信息准确, 避免用户误解。可能需要提醒用户注意订单状态的更新时间和联系客服的方式。

...

Thinking: RadixAttention 在这里的作用是什么?

第一轮对话时, 系统 Prompt + 用户消息的 KV Cache 被计算并存入 Radix 树。

第二轮对话时, 前面不变的部分 (系统 Prompt + 第一轮对话) 直接从 Radix 树命中缓存, 只需要计算新增的用户消息部分。

轮次越多, 缓存复用越多, 首 Token 延迟可降低 50% 以上。

SGLang 场景4：批量任务处理

场景4：批量任务处理（共享前缀省成本）

用同一个 Prompt 模板处理大量数据 -- 这是 RadixAttention 的最强场景，所有请求共享模板前缀的 KV Cache。

```
import concurrent.futures
from openai import OpenAI

client = OpenAI(base_url="http://localhost:8001/v1", api_key="not-needed")

# 待处理的文章列表
articles = [
    "人工智能正在改变医疗行业，通过深度学习算法，AI可以辅助医生进行影像诊断...",
    "区块链技术在供应链管理中的应用越来越广泛，它提供了透明的追溯机制...",
    "量子计算的突破使得传统加密算法面临挑战，后量子密码学成为研究热点...",
    # ... 更多文章
```

```
# 共享的评分 Prompt 模板（这部分的 KV Cache 只计算一次）
SYSTEM_PROMPT = """你是一个专业的文章评审员。请从以下三个维度评估文章：

1. 清晰度（1-10分）：逻辑是否清晰，表达是否流畅
2. 深度（1-10分）：是否有深入分析，不是泛泛而谈
3. 实用性（1-10分）：对读者是否有实际参考价值

请用 JSON 格式返回评分结果。"""

def score_article(article):
    response = client.chat.completions.create(
        model="Qwen3-0.6B",
        ...
```

4-批量任务处理.py

SGLang 场景4：批量任务处理

文章 1 评分: <think>

好的，我现在需要评估用户提供的文章的三个维度：清晰度、深度和实用性。首先，我得仔细阅读用户给的文章内容，确保自己理解每个部分。...

文章 2 评分: <think>

好的，我现在需要评估用户提供的文章的三个维度：清晰度、深度和实用性。首先，我得仔细阅读用户给的文章内容，理解其内容和结 ...

文章 3 评分: <think>

好的，我现在需要评估用户提供的这篇文章的三个维度：清晰度、深度和实用性。首先，我得仔细阅读用户给的文章内容，理解其核心信息 ...

性能对比：如果用 vLLM 处理 100 篇文章，系统 Prompt（约 100 tokens）被重复计算 100 次。SGLang 通过 RadixAttention 只计算 1 次，相当于省了 9900 tokens 的 Prefill 计算量。文章越多，节省越大。

SGLang 场景5：JSON 约束输出

场景5：JSON 约束输出（对接下游系统）

当你需要 LLM 输出严格格式的 JSON 给下游系统消费时，SGLang 的约束解码是最可靠的方案。

```
import requests
import json
# 使用 SGLang 原生 API 进行约束解码（通过 regex 参数）
response = requests.post(
    "http://localhost:8001/generate",
    json={ "text": "Extract the person's information from the following
text.\n\nText: My name is Zhang Wei, I am 28 years old, I work as a
software engineer at ByteDance.\n\nOutput:",
    "sampling_params": {
        "max_new_tokens": 200,
        "temperature": 0,
        "regex": r'\{"name": "[^"]+", "age": \d+, "company": "[^"]+", "role":
"[^"]+"\}'
```

```
    }
}
)
result = json.loads(response.json()["text"])
print(json.dumps(result, ensure_ascii=False, indent=2))
# 输出一定是合法 JSON:
# {"name": "Zhang Wei", "age": 28, "company": "ByteDance", "role":
"software engineer"}
```

```
{
  "name": "Zhang Wei",
  "age": 28,
  "company": "ByteDance",
  "role": "software engineer"
}
```

5-JSON 约束输出.py

Summary

约束解码 vs Prompt 工程

对比维度	Prompt 工程（传统方式）	约束解码（SGLang）
格式正确率	~90-95%（总有意料外）	100%（物理上不可能输出非法格式）
错误处理	需要 try-except + 重试逻辑	不需要，一次生成就是合法的
生产稳定性	凌晨3点可能因格式错误报警	睡觉安心

性能验证与监控

```
import time
import requests

def test_performance(prompt, n_requests=10):
    """简单的性能测试：测量 TTFT 和总延迟"""
    url = "http://localhost:8001/v1/completions"
    latencies = []

    for i in range(n_requests):
        start = time.time()
        response = requests.post(url, json={
            "model": "Qwen3-0.6B",
            "prompt": prompt,
            "max_tokens": 100,
            "temperature": 0.7
        })
        elapsed = time.time() - start
        latencies.append(elapsed)
```

```
avg_latency = sum(latencies) / len(latencies)
print(f"平均延迟: {avg_latency*1000:.0f}ms")
print(f"最小延迟: {min(latencies)*1000:.0f}ms")
print(f"最大延迟: {max(latencies)*1000:.0f}ms")

# 测试 1: 冷启动 (第一次请求, 无缓存)
print("=== 冷启动测试 ===")
test_performance("请解释什么是机器学习", n_requests=1)

# 测试 2: 缓存命中 (重复相同前缀的请求)
print("=== 缓存命中测试 ===")
test_performance("请解释什么是机器学习", n_requests=10)
```

性能验证与监控

```
# 测试 3: 批量共享前缀
print("=== 共享前缀测试 ===")
import concurrent.futures
prompts = [f"你是一个专业的AI助手。请回答以下问题：问题{i}" for
i in range(20)]
start = time.time()
with concurrent.futures.ThreadPoolExecutor(max_workers=10) as ex:
    futures = [ex.submit(
        requests.post, "http://localhost:8001/v1/completions",
        json={"model": "Qwen3-0.6B", "prompt": p, "max_tokens": 50}
    ) for p in prompts]
    [f.result() for f in futures]
print(f"20 个请求总耗时: {(time.time()-start)*1000:.0f}ms")
```

=== 冷启动测试 ===

平均延迟: 298ms

最小延迟: 298ms

最大延迟: 298ms

=== 缓存命中测试 ===

平均延迟: 208ms

最小延迟: 206ms

最大延迟: 213ms

=== 共享前缀测试 ===

20 个请求总耗时: 381ms

查看 SGLang 运行时指标

```
# 查看 Prometheus 格式的监控指标
curl http://localhost:8001/metrics

# 关注的关键指标：

# sglang:num_running_requests - 当前正在运行的请求数
# sglang:num_waiting_requests - 等待队列中的请求数
# sglang:token_usage          - Token 使用统计
# sglang:cache_hit_rate       - 缓存命中率（RadixAttention 的核心指标）
```

缓存命中率是 SGLang 的灵魂指标

cache_hit_rate 越高，说明 RadixAttention 复用的 KV Cache 越多，性能提升越明显。

- 多轮对话场景：命中率通常 60-80%
- 批量任务（共享模板）：命中率通常 80-95%
- 完全随机的单轮问答：命中率接近 0%（此时 SGLang 优势不明显）

打卡：SGLang部署与使用



对开源大模型进行私有化部署，选择适合的大模型，比如：

- deepseek-r1:1.5b, 7b, 8b, 14b, 32b, 70b等
- 或者qwen3系列：0.6b, 1.7b, 4b, 8b, 14b, 30b, 32b等
- 使用SGLang进行部署

部署任意的模型，调通即可

vLLM vs SGLang 性能对比实战

vLLM vs SGLang 性能对比实战

实验环境：

- GPU：RTX 4090 (24GB)
- 模型：Qwen3-0.6B
- vLLM：端口 8000
- SGLang：端口 8001

```
# 启动 vLLM (端口 8000)
```

```
vllm serve /root/autodl-tmp/models/qwen/Qwen3-0.6B \  
--enforce-eager \  
--gpu-memory-utilization 0.4
```

```
# 启动 SGLang (端口 8001)
```

```
python -m sglang.launch_server \  
--model-path /root/autodl-tmp/models/qwen/Qwen3-0.6B \  
--port 8001 --mem-fraction-static 0.4
```

同机部署注意：两个引擎运行在同一台机器，GPU 显存各分配 40%（--gpu-memory-utilization 0.4），确保不互相抢占资源。

CASE：vLLM 基线测试

先单独测试 vLLM 的基线性能，覆盖缓存、共享前缀、多轮对话、并发吞吐四个维度。

vLLM 基线测试结果

测试项	结果	说明
冷启动延迟	1482ms	首次请求，无 KV Cache
缓存命中延迟	1061ms (10次平均)	加速比 1.40x，Prefix Caching 生效
共享前缀 (20请求)	8722ms	平均每请求 436ms
多轮对话	第1轮 2183ms / 第2轮 1895ms / 第3轮 1907ms	后续轮次有缓存加速但幅度有限
并发吞吐 (8并发)	QPS 6.41 / 吞吐 641.3 tok/s	20个请求，总耗时 3.12s

CASE： SGLang 性能测试

使用相同的测试逻辑测试 SGLang，重点观察 RadixAttention 带来的缓存加速效果。

SGLang 性能测试结果

测试项	结果	说明
冷启动延迟	315ms	首次请求即极快
缓存命中延迟	222ms (10次平均)	最低 206ms， RadixAttention 高效复用
共享前缀 (20请求)	432ms	平均每请求仅 21.6ms

CASE: vLLM 与 SGLang 直接对比

在同一脚本中同时连接两个引擎，确保完全相同的请求、参数和测试条件，结果最具可比性。

```
# 同时连接两个引擎，保证公平对比
VLLM_URL = "http://localhost:8000/v1"
SGLANG_URL = "http://localhost:8001/v1"

# 三组对比测试
for name, url in {"vLLM": VLLM_URL, "SGLang": SGLANG_URL}.items():
    results["single"] = benchmark_single(url, n_requests=10)    # 单请求延迟
    results["shared_prefix"] = benchmark_shared_prefix(url)      # 共享前缀 (20请求)
    results["concurrent"] = benchmark_concurrent(url, concurrency=8) # 并发吞吐 (8并发)
```

CASE：vLLM 与 SGLang 直接对比

直接对比结果（同一脚本、同一时段）

指标	vLLM	SGLang	对比
单请求延迟 (ms)	1051.8	209.7	SGLang 快 5.0x
共享前缀耗时 (ms) 20请求并发	1071.2	378.9	SGLang 快 2.8x
QPS (8并发)	6.47	22.43	SGLang 高 3.5x
吞吐 (tok/s)	647.2	2242.5	SGLang 高 3.5x

Summary

Thinking: 为什么 SGLang 全面领先?

- 单请求延迟 (5.0x) :

vLLM 使用 `--enforce-eager` 且 Qwen3 默认开启 thinking 模式, 生成的 token 数远多于 SGLang (SGLang 未触发 thinking), 导致延迟差异被放大

- 共享前缀 (2.8x) :

SGLang 的 RadixAttention 通过 Radix 树自动识别 20 个请求的共同前缀, KV Cache 只计算一次; vLLM 的 Prefix Caching 是块级匹配, 粒度不如 Radix 树灵活

- 并发吞吐 (3.5x) :

SGLang 的缓存感知调度会优先处理命中率高的请求, 叠加 RadixAttention 减少重复计算, 整体 QPS 和吞吐量大幅领先

Summary

公平性说明:

- 两个引擎均使用 40% GPU 显存 (`--gpu-memory-utilization 0.4`) , 相同模型、相同硬件
- vLLM 使用了 `--enforce-eager` (禁用 `torch.compile`) , SGLang 使用默认配置
- Qwen3 模型在 vLLM 中默认触发了 thinking 模式 (生成更多 token) , 这是延迟差异偏大的重要原因
- 实际生产中差距可能因场景和配置不同而变化, 本实验重在对比趋势和验证 RadixAttention 的价值

Summary

vLLM 适合的场景

- 简单单轮问答，缓存价值不大
- 需要 EAGLE/MEDUSA 推测解码
- 与现有 OpenAI API 代码无缝迁移
- 社区生态和文档更成熟

SGLang 适合的场景

- 多轮对话（客服/面试），缓存命中率高
- 批量任务（共享前缀），成本大幅降低
- 结构化输出（JSON 约束解码）
- 复杂 LLM 程序（fork/join 并行）

选型决策

Thinking: 什么时候用 SGLang, 什么时候用 vLLM?

⇒先思考: 你的业务场景是什么?

简单问答 / 单轮对话

用 vLLM

成熟稳定, API 兼容性好

多轮对话 / 客服系统

用 SGLang

RadixAttention 缓存加速

批量任务 / 数据处理

用 SGLang

共享前缀省 3 倍成本

严格 JSON / Agent

用 SGLang

约束解码 100% 合法输出



Thank You
Using data to solve problems